

05. Heaps, trees, design

This chapter is about three things a robot program needs all the time: a way to always grab the biggest (or smallest) item quickly (a heap), a way to walk a tree of pointers (a binary tree), and a way to keep only the most recently used things in memory (a cache). A planner picks the cheapest cell next; a sensor filter needs the biggest reading in the last k samples; a map viewer keeps only the last few tiles in RAM. Every one of those is a problem below.

Rough time: 5 days at about 2 hours a day. Day 1 and 2 are Part A only. Do the problems only when you can draw a heap push and a level-order loop from memory.

What you need before this chapter. From chapter 01: `std::vector`, `std::pair`, `auto [a, b]` and `struct`. From chapter 03: `std::queue`, `std::stack`, recursion, and the ring-by-ring BFS loop. If the BFS ring loop is fuzzy, reread chapter 03 A2 first; the tree part of this chapter reuses it.

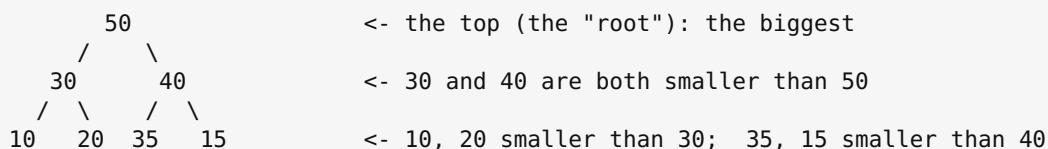
Part A — Heaps, trees and caches, from zero

A1. What a heap is

The picture. A heap is a pile where the biggest is always on top, and you only ever look at the top. Think of a stack of boxes arranged so that every box is bigger than the two boxes resting on it. You never search the pile. You ask "what is the biggest?" and the answer is sitting on top. You take it off, the pile shuffles a little, and the next biggest is on top.

That is the only promise a heap makes: **every parent is bigger than (or equal to) its two children**. It does not promise that the left child is bigger than the right, or that anything is sorted. Only "parent beats children", all the way down.

The drawing. Here is a heap of 7 numbers drawn as a tree:



Check the rule: 50 beats 30 and 40. 30 beats 10 and 20. 40 beats 35 and 15. It holds everywhere. Notice 35 is bigger than 30 even though 35 is lower down. That is allowed: the rule only compares a node with its own children.

The trick: the tree lives in an ordinary array. There are no pointers. We number the nodes top to bottom, left to right, starting at 0, and store them in a plain array in that order:

index:	0	1	2	3	4	5	6
value:	[50	, 30	, 40	, 10	, 20	, 35	, 15]

| | | | | |
 root children of 1 children of 2
 | |
 children of 0

The index arithmetic is the whole secret:

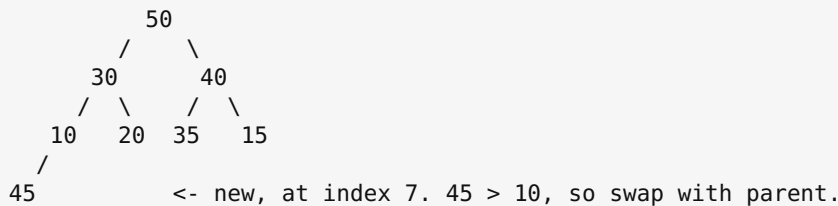
- the children of index i are at $2*i + 1$ and $2*i + 2$,
- the parent of index i is at $(i - 1) / 2$ (integer division, so the remainder is dropped).

Check it on the picture. Children of index 1 (value 30): $2*1+1 = 3$ and $2*1+2 = 4$, which hold 10 and 20. Correct. Children of index 2 (value 40): indices 5 and 6, values 35 and 15. Correct. Parent of index 5 (value 35): $(5-1)/2 = 2$, value 40. Correct. Parent of index 4: $(4-1)/2 = 3/2 = 1$ (integer division), value 30. Correct.

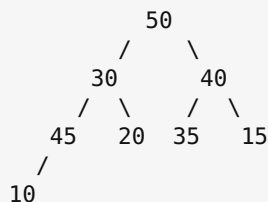
So "go to my parent" is one subtraction and one division, and "go to my left child" is one multiply and one add. That is why a heap is so fast: it is an array with a little arithmetic on top.

Push: put a new number in (sift up). Add the new number at the very end of the array (the next free spot in the bottom row), then let it climb: while it is bigger than its parent, swap with the parent. This is called *sift up*.

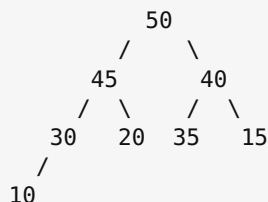
Push 45 into the heap above. Step 1: append at index 7. Its parent is $(7-1)/2 = 3$ (value 10).



Step 2: after the swap, 45 is at index 3. Its parent is $(3-1)/2 = 1$ (value 30). $45 > 30$, swap again.



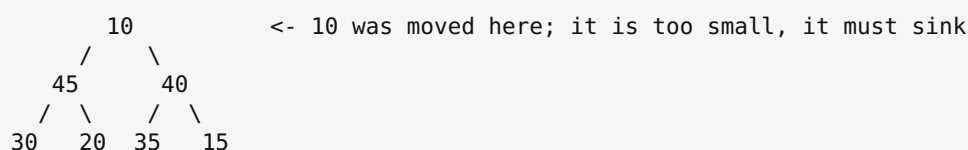
Step 3: 45 is at index 1. Its parent is index 0 (value 50). $45 < 50$, so stop.



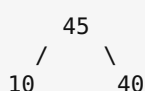
The array is now `[50, 45, 40, 30, 20, 35, 15, 10]`. Check the rule at every node: it holds.

Pop: take the biggest off (sift down). You want to remove the root. But removing index 0 would leave a hole at the top of the array. So: copy the *last* element into the root's slot, shrink the array by one, and let that element sink: while it is smaller than the bigger of its two children, swap with that bigger child. This is *sift down*.

Pop from the heap we just built. Step 1: the last element is 10 (index 7). Move it to index 0, drop the last slot. 50 is the answer we return.



Step 2: children of index 0 are 45 and 40. The bigger is 45. $10 < 45$, swap.



```

      /  \    /  \
    30   20 35   15

```

Step 3: 10 is at index 1. Children are 30 and 20. Bigger is 30. $10 < 30$, swap.

```

      45
     /  \
    30   40
   /  \  /  \
  10  20 35  15

```

Step 4: 10 is at index 3. Its children would be at 7 and 8, which do not exist. Stop. The array is [45, 30, 40, 10, 20, 35, 15] and 45, the second biggest, is on top. Exactly what we want.

Why push and pop cost $\log n$. Look at how the levels grow: level 0 has 1 node, level 1 has 2, level 2 has 4, level 3 has 8. Each level doubles. So a heap with n nodes has about $\log_2(n)$ levels (7 nodes: 3 levels; 8 nodes: 4 levels; 1000 nodes: 10 levels; a million nodes: 20 levels). Sift up moves at most one level per swap and sift down moves at most one level per swap, so each is at most $\log_2(n)$ swaps. A million elements, 20 swaps. That is what " $O(\log n)$ " means here: the height of the tree.

The Python you would have written. Python's `heapq` is a heap on a list, but it keeps the *smallest* on top (a min-heap). For "biggest on top" people push negatives.

```

import heapq
h = []
for x in [50, 30, 40, 10, 20, 35, 15]:
    heapq.heappush(h, x)      # smallest on top
print(h[0])                  # 10
print(heapq.heappop(h))      # 10
print(h[0])                  # 15

```

The same thing in C++. `std::priority_queue` is the heap. The biggest is on top by default.

```

#include <queue>           // std::priority_queue lives here
#include <functional>      // std::greater

std::priority_queue<int> pq;           // biggest on top (a max-heap)
pq.push(30);
pq.push(50);
pq.push(40);
int biggest = pq.top();                // 50: look at the top
pq.pop();                             // remove it; now 40 is on top
bool e = pq.empty();                  // false

```

- `std::priority_queue<int> pq;` says "a heap of ints, biggest on top". The name is "priority queue" because the item with the highest priority always comes out first.
- `push(x)` is `heapq.heappush`. `top()` is `h[0]`. `pop()` removes the top and, as with every C++ container, returns nothing; look with `top()` first, then `pop()`.
- `empty()` and `size()` are the same as on `std::queue`.

For **smallest on top** you add two more things inside the angle brackets:

```

std::priority_queue<int, std::vector<int>, std::greater<int>> minq; // smallest on top
minq.push(30); minq.push(50); minq.push(10);
int smallest = minq.top();      // 10

```

- The second thing, `std::vector<int>`, is the array the heap lives in. You almost always write exactly this.
- The third thing, `std::greater<int>`, is the comparison the heap uses. The default is `std::less<int>` and gives "biggest on top". `std::greater<int>` flips it to "smallest on top". Memorise this line as one unit; everyone types it the same way.

A heap of pairs. Very often you want to sort by one number but carry another along, for example (distance, id). Put the number you want to sort by *first* in the pair. Pairs compare by the first element, and only look at the second on a tie.

```
using P = std::pair<double, int>; // (distance, id)
std::priority_queue<P, std::vector<P>, std::greater<P>> pq; // smallest distance on top
pq.push({2.5, 7});
pq.push({0.5, 3});
pq.push({1.0, 9});
int nearestId = pq.top().second; // 3
```

- `using P = std::pair<double, int>;` gives the long type a short name so the next line is readable. It is like a Python alias `P = tuple`.
- `.first` and `.second` are the two halves of a pair. `{0.5, 3}` builds one.

(You can skip this on a first read.) `top()` on an empty priority queue is undefined behaviour, not an exception: check `empty()` first. `top()` also hands you a `const` reference, so you cannot change the top element in place; pop it, change it, push it back. If you want your own ordering on a struct, you write a small function (a lambda) that answers "should a come out *after* b?" and pass it as the third thing; `00_heap_tree_toolkit.cpp` shows the exact spelling with `decltype(cmp)` because it is fiddly to remember.

Try it. In `00_heap_tree_toolkit.cpp`, the `MaxHeap` struct is the sift-up and sift-down above in about 30 lines. Change `data` in `main` to `{50, 30, 40, 10, 20, 35, 15, 45}` and write down what `h.a` looks like after all the pushes before you run it. (Hint: the trace above.)

In your own words: "A heap is a tree stored in an array where every parent beats its children, so the biggest is always at index 0. Children of i are at $2i+1$ and $2i+2$. Push appends and sifts up, pop moves the last element to the root and sifts down; both cost the height, $\log n$. In C++ it is `std::priority_queue`, biggest on top by default, `std::greater` for smallest."

A2. The "top k" patterns: a heap of size k, and nth_element

Many questions ask for "the k largest", "the k closest", "the kth biggest". Two tools.

Tool 1: a small heap of size k. Picture a VIP list with k seats. Newcomers only get in if they beat the *worst* VIP currently seated, and when they get in, the worst VIP leaves. At the end the k seats hold the k best people, and the person in the worst seat is exactly the "kth best".

The heap trick: to find the k *largest*, keep a *min*-heap (smallest on top) of size k. That sounds backwards, but the top of a min-heap is the *worst* of the VIPs, which is exactly the one you need to compare newcomers against and throw out.

Find the 3 largest of `[7, 2, 9, 4, 8]`, so $k = 3$. The heap is drawn as a set with its top underlined.

see 7:	heap {7}	fewer than 3 seats used, just add
see 2:	heap {2, 7}	top is 2
see 9:	heap {2, 7, 9}	3 seats used, top is 2
see 4:	push 4 -> {2, 4, 7, 9}	4 people, one too many
	pop top -> {4, 7, 9}	2 was the worst, 2 leaves
see 8:	push 8 -> {4, 7, 8, 9}	one too many again
	pop top -> {7, 8, 9}	4 leaves

The heap holds `{7, 8, 9}`: the 3 largest. Its top, 7, is the 3rd largest. Each push and pop costs $\log k$, not $\log n$, because the heap never has more than $k+1$ things in it. For n numbers that is $n \log k$ work, and you only ever hold k numbers in memory. This is the tool when numbers arrive one at a time from a sensor and you cannot store them all.

The rule to remember: **k largest needs a min-heap, k smallest needs a max-heap**. The top is always "the worst of the best" so it can be thrown out.

In Python:

```
import heapq
def k_largest(nums, k):
    h = []
    for x in nums:
        heapq.heappush(h, x)
        if len(h) > k:
            heapq.heappop(h)    # throw out the smallest of the k+1
    return sorted(h)          # [7, 8, 9] for [7, 2, 9, 4, 8], k = 3
```

Tool 2: `std::nth_element`, "partially sort just enough to know who is at position k". If you sorted the whole array, position $k-1$ (counting from 0) would hold the k th smallest. But sorting everything is wasted work; you only care about one position. `std::nth_element` reorders the array *just enough* so that the element at position $k-1$ is the one that would be there after a full sort, everything before it is smaller or equal, and everything after it is bigger or equal. Nothing else is sorted.

```
before: [9, 2, 7, 4, 5, 1, 8, 3, 6]    want the 3rd smallest, position 2
after:  [1, 2, 3, ?, ?, ?, ?, ?, ?]    position 2 holds 3 (the true 3rd smallest)
      ^^^ these are <= 3             ^^^^^^^^^^^^^^^^^ these are >= 3, in no particular order
```

```
#include <algorithm>    // std::nth_element
std::vector<int> v = {9, 2, 7, 4, 5, 1, 8, 3, 6};
int k = 3;
std::nth_element(v.begin(), v.begin() + (k - 1), v.end());
int third = v[k - 1];    // 3
```

- `std::nth_element(first, nth, last)` takes three positions: where the range starts, the one position you care about, and one past the end. `v.begin() + (k - 1)` is "position $k-1$ ".
- It changes the order of `v`. If you need the original order later, copy first.
- On average it does a small constant times n steps (it is a "quickselect": it partitions like quicksort but only recurses into the half that contains position k). Python has no built-in for this; `sorted(v)[k-1]` is the usual substitute and costs $n \log n$.

When to use which.

Situation	Use	Why
Numbers arrive one at a time (a stream), you cannot hold them all	heap of size k	Only k things in memory, $\log k$ per number
Everything is in a vector and you may reorder it	<code>std::nth_element</code>	Average n steps, no extra memory
k is close to n	just <code>std::sort</code>	The heap would be nearly $n \log n$ anyway
" k most frequent"	count into a hash map first, then either tool on the counts	The counting is the real work

In your own words: "For the k largest I keep a min-heap of size k ; its top is the worst of the best, so a newcomer only gets in if it beats that. That is $n \log k$ time and k memory, and it works on a stream. If all the data is in a vector I can reorder, `std::nth_element` puts the k th element in its sorted position in linear time on average."

A3. Monotonic deque and monotonic stack

Both of these are lists where you throw away things that can never be the answer again. The word "monotonic" just means the values in the list only go one way (always decreasing from front to back, say). That property is what makes them fast.

The problem the deque solves: sliding window maximum. You have a list of numbers and a window of width k that slides along it one step at a time. For every position of the window you want the biggest number inside it.

```
nums = [2, 5, 1, 4, 3], k = 3

[2, 5, 1] 4 3    -> max 5
2 [5, 1, 4] 3    -> max 5
2 5 [1, 4, 3]    -> max 4      answer: [5, 5, 4]
```

The slow way is to look at all k numbers every time: n times k work. The fast way keeps a short list of *candidates* and does a constant amount of work per number.

The picture. Imagine people standing in a line and the window is a viewing box that slides over them. You want the tallest person in the box. Here is the key thought: *a shorter person standing behind a taller one can never be the tallest in the window while the taller one is there*. The taller one entered later, so it leaves the window later. Until it leaves, the shorter person in front of it is useless. So the moment a new tall person arrives, we throw out every shorter person who arrived before them.

That leaves a list of people that is tallest at the front and shorter towards the back: a *monotonic decreasing* list. The front is always the answer. And when the front person walks out of the window (their index is too old), we drop them from the front.

We store *indices* (positions), not values, because we need the index to know when someone has left the window. We can always look up the value with `nums[index]`.

The trace. Deque shown as `[index(value), ...]`, front on the left. The window covers indices $i-k+1 \dots i$.

<code>i=0, value 2:</code>	deque empty, push 0.	deque [0(2)]	window not full yet
<code>i=1, value 5:</code>	back is 0(2), $2 \leq 5$, throw it out. push 1.	deque [1(5)]	window not full yet
<code>i=2, value 1:</code>	back is 1(5), $5 > 1$, keep it. push 2.	deque [1(5), 2(1)]	window is 0..2, answer <code>nums[1] = 5</code>
<code>i=3, value 4:</code>	front 1 still in window ($1 > 3-3=0$). back is 2(1), $1 \leq 4$, throw out. back is 1(5), $5 > 4$, keep. push 3.	deque [1(5), 3(4)]	window 1..3, answer 5
<code>i=4, value 3:</code>	front is 1, and $1 \leq 4-3 = 1$: too old, drop from front. back is 3(4), $4 > 3$, keep. push 4.	deque [3(4)] deque [3(4), 4(3)]	window 2..4, answer 4

Answers: `[5, 5, 4]`. Every index was pushed once and thrown out at most once, so the total work is about $2n$, no matter how the inner "throw out" loop looks.

Why a deque and not a stack or queue. We drop from the front (too old) and from the back (too short), and push at the back. A container that is fast at both ends is a *double-ended queue*, "deque" for short. Python has `collections.deque`; C++ has `std::deque` with `push_back`, `pop_back`, `front`, `pop_front`, `back`.

The Python and the C++ for this are Problem 4 below. Read the trace again; then the code will look obvious.

The stack version: next greater element. A close cousin. For each number, what is the first number to its right that is bigger? If there is none, say -1.

```
nums = [2, 1, 4, 3]
answer = [4, 4, -1, -1]      (2 -> 4, 1 -> 4, 4 -> nothing, 3 -> nothing)
```

Same thought, but now nothing ever "leaves a window", so only the back end matters, which makes it a stack. Walk left to right. Keep a stack of indices whose values are still *waiting* for their bigger number, tallest at the bottom. When a new number arrives, every waiting number smaller than it has just found its answer: pop them and write the answer. Then the new number starts waiting.

i=0, value 2:	stack empty. push 0.	stack [0(2)]
i=1, value 1:	top 2 < 1? no. push 1.	stack [0(2), 1(1)]
i=2, value 4:	top 1 < 4: answer[1] = 4, pop.	
	top 2 < 4: answer[0] = 4, pop.	
	push 2.	stack [2(4)]
i=3, value 3:	top 4 < 3? no. push 3.	stack [2(4), 3(3)]
end:	2 and 3 are still waiting: answer stays -1.	

The Python:

```
def next_greater(nums):
    ans = [-1] * len(nums)
    stack = [] # indices, values decreasing bottom to top
    for i, x in enumerate(nums):
        while stack and nums[stack[-1]] < x:
            ans[stack.pop()] = x
        stack.append(i)
    return ans # [4, 4, -1, -1] for [2, 1, 4, 3]
```

The C++:

```
#include <vector>
std::vector<int> nextGreater(const std::vector<int>& a) {
    std::vector<int> ans(a.size(), -1); // same size as a, all -1
    std::vector<int> st; // the stack: indices, values decreasing bottom to top
    for (int i = 0; i < static_cast<int>(a.size()); ++i) {
        while (!st.empty() && a[st.back()] < a[i]) {
            ans[st.back()] = a[i]; // the waiting index just found its bigger number
            st.pop_back();
        }
        st.push_back(i);
    }
    return ans;
}
```

- `std::vector<int> ans(a.size(), -1)` is Python's `[-1] * len(nums)`: "a vector of that many copies of -1".
- We use a `std::vector` as the stack: `push_back` is `append`, `back()` is `stack[-1]`, `pop_back()` is `pop()`. `std::stack` would also work; `vector` is just as easy and lets you look inside while debugging.
- `while (!st.empty() && ...)`: the `!st.empty()` check must come first. `st.back()` on an empty vector is undefined behaviour in C++, where Python would raise `IndexError`.

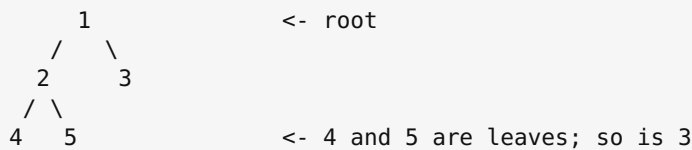
Trapping Rain Water (Problem 8) has a monotonic stack version, and so do "largest rectangle in a histogram" and "stock span". If a problem says "for each element, the nearest bigger/smaller one to the left/right", it is this.

Try it. Change `<` to `<=` in the `while` of `nextGreater` and run it on `[3, 3, 5]`. Predict the answer for the first 3 before you run it.

In your own words: "For a sliding window maximum I keep a deque of indices with values decreasing from front to back. A new value throws out every smaller value behind it, because those are older and smaller and can never win. The front is the window maximum, and I drop it when its index falls out of the window. Each index is pushed and popped once, so it is linear. Next-greater-element is the same idea with a stack."

A4. Binary trees in C++

The picture. A binary tree is a family tree where every node has at most two children, called left and right. The one at the top is the *root*. A node with no children is a *leaf*.



The node. In Python you would have had a small class with `val`, `left` and `right`. In C++ it is a `struct` with the same three fields, and it is the exact struct LeetCode gives you:

```

struct TreeNode {
    int val;           // the number stored in this node
    TreeNode* left;    // an arrow to the left child, or nullptr if there is none
    TreeNode* right;   // an arrow to the right child, or nullptr if there is none
    TreeNode() : val(0), left(nullptr), right(nullptr) {}
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
    TreeNode(int x, TreeNode* left, TreeNode* right) : val(x), left(left), right(right) {}
};
  
```

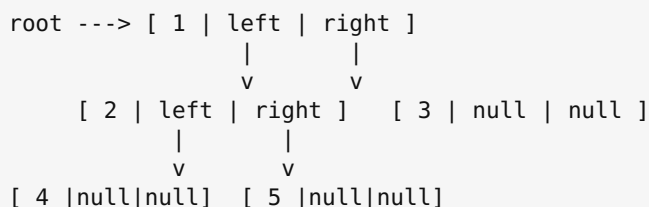
- `TreeNode* left;` is a *pointer*: the address of another node, like a house address written on a sticky note. An arrow in the drawing is a pointer in the code. `nullptr` means "no address, there is nothing there"; it is Python's `None`.
- The three lines starting `TreeNode(...)` are *constructors*: recipes for making a node. `: val(x), left(nullptr), right(nullptr)` fills the three fields. `new TreeNode(4)` makes a node holding 4 with no children and gives you its address.
- To follow an arrow you write `node->left` (Python `node.left`). The `->` means "go to the address stored in this pointer, then take that field".
- `if (node == nullptr)` or simply `if (!node)` is Python's `if node is None`.

Building the 5-node tree by hand:

```

TreeNode* root = new TreeNode(1);
root->left = new TreeNode(2);
root->right = new TreeNode(3);
root->left->left = new TreeNode(4);
root->left->right = new TreeNode(5);
  
```

In memory that is five separate boxes, joined by arrows:



The three traversals: when do you say the node's name? Walking a tree means visiting every node. The recursive shape is always "visit left subtree, visit right subtree", and the only choice is *when* you say the node's own name: before the children, between them, or after.

pre-order:	say ME, then left, then right	->	1 2 4 5 3
in-order:	left, then say ME, then right	->	4 2 5 1 3
post-order:	left, then right, then say ME	->	4 5 2 3 1

Trace in-order on the 5-node tree. Start at 1: first do the left subtree (rooted at 2). At 2: first do its left subtree (4). At 4: no left child, say **4**, no right child, done. Back at 2: say **2**. Now 2's right subtree (5): say **5**. Back at 1: say **1**. Then 1's right subtree (3): say **3**. Result 4 2 5 1 3.

Now pre-order: at 1 say **1** first, then go left: at 2 say **2**, go left: say **4**, back, go right: say **5**, back to 1, go right: say **3**. Result 1 2 4 5 3.

Post-order: at 1, do left first: at 2, do left first: at 4, nothing below, say **4**. Back at 2, do right: say **5**. Now 2 has finished both children, say **2**. Back at 1, do right: say **3**. Now 1 has finished both, say **1**. Result 4 5 2 3 1.

The Python:

```
def inorder(node, out):
    if node is None:
        return
    inorder(node.left, out)
    out.append(node.val)           # "say my name" in the middle
    inorder(node.right, out)
```

The C++:

```
void inorder(TreeNode* root, std::vector<int>& out) {
    if (root == nullptr) return;
    inorder(root->left, out);
    out.push_back(root->val);    // move this line to the top for pre-order, to the bottom for post-order
    inorder(root->right, out);
}
```

- `std::vector<int>& out`: the `&` means every recursive call appends to the *same* vector. Without it each call would get its own copy and the result would be empty. Same rule as the `visited` table in chapter 03.

Which one when? **Post-order** is the one you will use most: "finish the children, then use their results" is how you compute a height, delete a tree, or find a lowest common ancestor (A5). **Pre-order** is for copying or printing a tree. **In-order** on a *binary search tree* (left smaller, right bigger) gives the values in sorted order.

Level order: the queue and the ring loop. The fourth way to walk a tree is by depth: the root, then both its children, then all the grandchildren. That is BFS from chapter 03 with a queue, and it uses the same "snapshot the queue size, process exactly that many" loop that gave you the rings in Rotting Oranges.

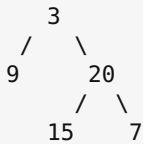
```
level 0: [1]
level 1: [2, 3]
level 2: [4, 5]
```

```
#include <queue>
std::vector<std::vector<int>> levelOrder(TreeNode* root) {
    std::vector<std::vector<int>> out;
    if (root == nullptr) return out;
    std::queue<TreeNode*> q;           // a queue of pointers to nodes
    q.push(root);
    while (!q.empty()) {
        int levelSize = static_cast<int>(q.size()); // how many nodes are on THIS level
        std::vector<int> level;
        for (int i = 0; i < levelSize; ++i) {
            TreeNode* cur = q.front();
            q.pop();
            level.push_back(cur->val);
            if (cur->left) q.push(cur->left);    // children go to the back: next level
            if (cur->right) q.push(cur->right);
        }
        out.push_back(level);
    }
}
```

```
    return out;
}
```

- `std::queue<TreeNode*>`: the queue holds addresses of nodes, not copies of nodes.
- `levelSize` is taken *before* the inner loop, because pushing children makes the queue grow while we are inside it. Read chapter 03 A2, Variant 1, if this is not clear.
- `if (cur->left)` is short for `if (cur->left != nullptr)`.

Building a tree from a list, for tests. LeetCode writes a tree as a list in level order, with `null` for a missing child: `[3, 9, 20, null, null, 15, 7]` means



Read it like this: 3 is the root. The next two entries, 9 and 20, are its children. The next two, `null, null`, are 9's children (none). The next two, 15 and 7, are 20's children. The builder is a small BFS: keep a queue of parents that still need children, and hand out two list entries to each parent in turn. In the test files `-1` stands for `null` and the function is called `buildTree`; you will see it in every tree problem's file. You do not need to memorise it, only to know how to read the list.

Freeing nodes. Every `new` should be matched by a `delete`, or the program leaks memory. Deleting a tree is a post-order walk: delete both children first, then yourself. If you deleted yourself first you would lose the arrows to the children and could never reach them.

```
void deleteTree(TreeNode* root) {
    if (root == nullptr) return;
    deleteTree(root->left);
    deleteTree(root->right);
    delete root;           // children are gone; now this node
}
```

LeetCode does not check for leaks, but the test files in this repo free every tree they build. Get into the habit.

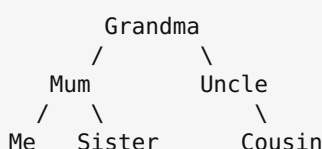
(You can skip this on a first read.) Recursion depth equals tree height. For a balanced tree of a million nodes that is 20 calls, no problem. For a tree that is one long chain (each node has only a right child) it is a million calls and the program crashes with a stack overflow. The fix is an explicit stack: `inorderIterative` in the toolkit file shows the in-order version (push the whole left spine, pop one, visit it, step right, repeat). Learn it after the recursive version is solid.

Try it. In `00_heap_tree_toolkit.cpp`, change the `preorder` function so that the `out.push_back` line is *after* the two recursive calls. Predict the vector it produces for the tree `{3, 9, 20, NONE, NONE, 15, 7}` before running (the assert will fail; that is expected).

In your own words: "A `TreeNode` is a value plus two pointers, left and right, `nullptr` when missing. Pre, in and post-order differ only in when I record the node: before, between, or after its children. Level order is BFS with a queue and the level-size snapshot loop. Deleting is post-order: children first, then delete root."

A5. Lowest common ancestor

The picture. A family tree:



The *lowest common ancestor* (LCA) of two people is the nearest person who is above both of them. For Me and Sister it is Mum. For Me and Cousin it is Grandma. For Mum and Me it is Mum herself: a person counts as their own ancestor in this problem.

The idea: ask both children, report what you found. Give every node one job. Someone hands it two names, p and q, and asks: "look in your subtree; what did you find?" A node answers with one of three things:

- "nothing": neither p nor q is anywhere below me, and I am not one of them;
- "p" (or "q"): I found exactly one of them, and here it is;
- "me": I found *both*, one on each side, so I am the meeting point, the LCA.

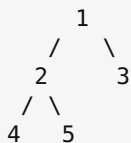
And there is one shortcut: if the node *is* p or q, it answers with itself straight away without looking further down. (If the other one is below it, this node is the answer anyway.)

A node computes its answer by asking its left child and its right child, then combining:

left says	right says	I say	
nothing	nothing	nothing	
p	nothing	p	(pass up whatever the one side found)
nothing	q	q	
p	q	ME	(both found on different sides: I am the LCA)
some node X	nothing	X	(a lower node already decided; pass it up unchanged)

Because a node answers only *after* both children have answered, this is a post-order walk from A4.

The trace. Use the 5-node tree from A4 and look for the LCA of 4 and 5.



```

ask 1: not 4 or 5. ask my children.
  ask 2: not 4 or 5. ask my children.
    ask 4: I AM 4. answer "4".
    ask 5: I AM 5. answer "5".
    2 hears "4" from the left and "5" from the right: both sides. answer "2" (I am the LCA).
  ask 3: not 4 or 5. ask children: both nullptr, both answer "nothing". answer "nothing".
1 hears "2" from the left and "nothing" from the right. pass up "2".
answer: 2
  
```

Now the LCA of 4 and 3:

```

ask 1: not 4 or 3. ask children.
  ask 2: not them. ask children.
    ask 4: I AM 4. answer "4".
    ask 5: not them; no children; answer "nothing".
    2 hears "4" and "nothing". pass up "4".
  ask 3: I AM 3. answer "3".
1 hears "4" from the left and "3" from the right: both sides. answer "1".
answer: 1
  
```

The Python:

```

def lca(root, p, q):
    if root is None or root is p or root is q:
        return root
    left = lca(root.left, p, q)
    right = lca(root.right, p, q)
    if left and right:
        # "nothing", or "me" if I am one of them
  
```

```

return root                                # found on both sides: I am the LCA
return left if left else right             # pass up whatever one side found

```

Note the `is`, not `==`: we compare *which node*, not what value it holds, because values can repeat. In C++ the same comparison is `root == p` on the pointers (two addresses are equal only if they are the same node). The C++ is Problem 6.

Every node is asked once, so this is one pass over the tree. The recursion goes as deep as the tree is tall.

In your own words: "Each node asks its two children what they found. Nothing from both means nothing. One side found something means pass it up. Both sides found something means I am the lowest common ancestor. A node that is `p` or `q` answers with itself immediately. It is a post-order walk, one pass, linear time."

A6. Design problems and the LRU cache

What a cache is. A cache is a small, fast storage that holds copies of the few things you used most recently, so you do not have to fetch them from the slow place (disk, network, a big computation) again. It has a fixed size, so when it is full and something new arrives, something old must go.

What "least recently used" means: the bookshelf story. You have a shelf with room for 3 books. Every time you use a book, you put it back at the *front* of the shelf. When you need a book that is not on the shelf, you get it from the library and put it at the front; if the shelf was full, the book at the *back* (the one you have not touched for the longest time) goes back to the library. That book is the "least recently used", and this rule is an LRU cache.

start (empty):	[		]			
use A:	[	A		]		
use B:	[	B	A	]		
use C:	[	C	B	A	]	full
use A again:	[	A	C	B	]	A moves to the front, nothing removed
use D (not on shelf):	[	D	A	C	]	shelf was full: B, at the back, is removed
use B (not on shelf):	[	B	D	A	]	C is removed

The cache in Problem 7 has two operations: `get(key)` returns the value stored under that key (or -1 if it is not there) and counts as "using" it, and `put(key, value)` stores a value (counts as using it too), removing the least recently used entry if the cache was full. Both must be fast no matter how big the cache is: a constant number of steps, not "walk along the shelf".

Why we need a hash map AND a linked list. Two questions must be answered quickly:

1. "Is key 7 on the shelf, and where?" Walking along the shelf is slow. A *hash map* (Python `dict`, C++ `std::unordered_map`) answers "where is key 7" in one step.
2. "Move this book to the front, and remove the one at the back." In an array (`std::vector`), moving a middle item to the front means shifting everything in between: slow. In a *doubly linked list*, every book has an arrow to the one before and the one after; moving a book means rewiring four arrows, no shifting. One step.

So we keep both, and they point at each other:

hash map (key -> where on the shelf)	the shelf: a doubly linked list
-----	front
key 1 ----->	[k=1,v=10] <--> [k=2,v=20] <--> [k=3,v=30]
key 2 ----->	^
key 3 ----->	^
	back

Each map entry stores a *bookmark* to that key's node in the list. Given a key, the map takes you straight to the node; from the node, the list lets you unhook it and rehook it at the front. And each list node stores its *key* as well as its value, so that when the back node is removed we know which map entry to delete too.

What `std::list::splice` does. In C++ the doubly linked list is `std::list`, and the bookmark is a `std::list iterator` (a pointer-like object that names one node). The move to the front is one call:

```
#include <list>
#include <unordered_map>

std::list<int> shelf = {1, 2, 3, 4}; // front is 1
std::unordered_map<int, std::list<int>::iterator> where; // key -> bookmark into the list
for (auto it = shelf.begin(); it != shelf.end(); ++it) where[*it] = it;

shelf.splice(shelf.begin(), shelf, where[3]); // move the node holding 3 to the front
// shelf is now {3, 1, 2, 4}; where[3] still points at the node holding 3
```

- `std::list<int>::iterator` is the type of a bookmark into a `std::list<int>`. `*it` is the value at the bookmark, `shelf.begin()` is the bookmark of the front node.
- `shelf.splice(pos, shelf, it)` means "unhook the node at `it` and rehook it just before `pos`". Nothing is copied and no memory is allocated; only the arrows change. That is why it is constant time.
- The stored bookmark `where[3]` is still valid after the splice, because the *node* did not move in memory, only its neighbours changed. This is the property that makes the whole design work. A `std::vector` would not give you this: adding to a vector can move every element to a new place in memory and every stored position becomes garbage.
- `shelf.erase(it)` removes one node, `shelf.pop_back()` removes the back, `shelf.back()` looks at the back, `shelf.emplace_front(key, value)` builds a new node at the front.

Python hides all of this in `OrderedDict.move_to_end`; C++ makes you build it, which is why the problem is popular.

How to attack a design problem. Write down the operations and what each returns, including the odd cases (put on a key that already exists, capacity 1). Then write down how fast each operation must be. Only then pick the containers; the speed requirement is what chooses them.

In your own words: "An LRU cache keeps the most recently used items and drops the least recently used when full. The hash map finds a key in one step; the doubly linked list moves a node to the front and drops the back in one step. The map stores list iterators as bookmarks, which stay valid because `std::list::splice` only rewires arrows. Each node also stores its key so eviction can erase the map entry."

A7. Where this shows up in robots

- **The heap is the planner's to-do list.** Dijkstra and A* (chapter 03, Part C) keep the cells still to be explored in a `std::priority_queue` of `(cost, cell)` pairs and always pop the cheapest. The pair-comparison rule from A1 is what makes ties break sensibly.
- **k nearest points.** Finding the k closest points in a laser scan or a point cloud is a bounded heap of size k (A2) exactly like Problem 2. Libraries like PCL use a tree to decide which points to *offer*, but the "keep the k best" part is this heap.
- **Sliding window filters.** "The biggest range reading in the last k samples" for spike rejection, computed at 1 kHz, is the monotonic deque of A3: constant work per sample.
- **Tile and submap caches.** A map viewer or a localiser keeps only the last N map tiles in RAM; that is an LRU cache (A6).
- **Trees.** Collision-checking hierarchies, octrees for 3D occupancy, and the tf frame tree in ROS are all trees; level order and post-order walks are how you print, prune, or compute bounds on them, and `LookupTransform` between two frames is an LCA query (A5).

A8. The toolkit file

code/05-heaps-trees-design/00_heap_tree_toolkit.cpp has everything from A1 to A6 in one file with asserts: a hand-written max-heap, the standard heap functions on a vector, `std::priority_queue` in its three spellings, `nth_element`, `TreeNode` with `buildTree` and the traversals, and `std::list::splice` with a map of bookmarks.

```
// Chapter 05 toolkit demo: hand-rolled binary heap, std heap algorithms,
// nth_element, TreeNode build-from-vector + traversals, std::list::splice.
#include <algorithm>
#include <cassert>
#include <functional>
#include <iostream>
#include <list>
#include <queue>
#include <string>
#include <unordered_map>
#include <vector>

// ---- 1. A max-heap by hand on a std::vector -----
// Layout: node i has children 2i+1 and 2i+2, parent (i-1)/2. No pointers, no
// allocation per node, cache-friendly – that is why heaps live in arrays.
struct MaxHeap {
    std::vector<int> a;

    void push(int x) {
        a.push_back(x);
        siftUp(static_cast<int>(a.size()) - 1);
    }
    int top() const { return a.front(); }
    void pop() {
        std::swap(a.front(), a.back()); // move last leaf to the root ...
        a.pop_back();
        if (!a.empty()) siftDown(0);    // ... and let it sink to its place
    }
    bool empty() const { return a.empty(); }

    void siftUp(int i) {
        while (i > 0) {
            int p = (i - 1) / 2;
            if (a[p] >= a[i]) break;    // heap property restored
            std::swap(a[p], a[i]);
            i = p;
        }
    }
    void siftDown(int i) {
        int n = static_cast<int>(a.size());
        while (true) {
            int l = 2 * i + 1, r = 2 * i + 2, best = i;
            if (l < n && a[l] > a[best]) best = l;
            if (r < n && a[r] > a[best]) best = r;
            if (best == i) break;
            std::swap(a[i], a[best]);
            i = best;
        }
    }
};

// ---- 2. TreeNode as LeetCode defines it + level-order builder -----
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode() : val(0), left(nullptr), right(nullptr) {}
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
    TreeNode(int x, TreeNode* left, TreeNode* right) : val(x), left(left), right(right) {}
};
```

```

// Builds from LeetCode's level-order encoding; NONE marks a missing child.
// Children are only queued for non-null nodes, exactly like LeetCode's format.
const int NONE = -1;
TreeNode* buildTree(const std::vector<int>& vals) {
    if (vals.empty() || vals[0] == NONE) return nullptr;
    TreeNode* root = new TreeNode(vals[0]);
    std::queue<TreeNode*> q;
    q.push(root);
    int i = 1;
    int n = static_cast<int>(vals.size());
    while (!q.empty() && i < n) {
        TreeNode* cur = q.front();
        q.pop();
        if (i < n && vals[i] != NONE) { cur->left = new TreeNode(vals[i]); q.push(cur->left); }
        ++i;
        if (i < n && vals[i] != NONE) { cur->right = new TreeNode(vals[i]); q.push(cur->right); }
        ++i;
    }
    return root;
}

void deleteTree(TreeNode* root) {           // post-order: children first, then self
    if (root == nullptr) return;
    deleteTree(root->left);
    deleteTree(root->right);
    delete root;
}

void inorder(TreeNode* root, std::vector<int>& out) {
    if (root == nullptr) return;
    inorder(root->left, out);
    out.push_back(root->val);
    inorder(root->right, out);
}

void preorder(TreeNode* root, std::vector<int>& out) {
    if (root == nullptr) return;
    out.push_back(root->val);
    preorder(root->left, out);
    preorder(root->right, out);
}

// Inorder without recursion: the explicit stack replaces the call stack.
std::vector<int> inorderIterative(TreeNode* root) {
    std::vector<int> out;
    std::vector<TreeNode*> stack;
    TreeNode* cur = root;
    while (cur != nullptr || !stack.empty()) {
        while (cur != nullptr) { stack.push_back(cur); cur = cur->left; }
        cur = stack.back();
        stack.pop_back();
        out.push_back(cur->val);
        cur = cur->right;
    }
    return out;
}

int main() {
    // ---- hand-rolled heap vs std::sort ----
    {
        std::vector<int> data = {5, 1, 9, 3, 7, 3, 8, 0, -2, 6};
        MaxHeap h;
        for (int x : data) h.push(x);
        std::vector<int> popped;
        while (!h.empty()) { popped.push_back(h.top()); h.pop(); }
        std::vector<int> sorted = data;
        std::sort(sorted.begin(), sorted.end(), std::greater<int>());
        assert(popped == sorted);
    }

    // ---- std::make_heap / push_heap / pop_heap on a vector ----
    {
        std::vector<int> v = {4, 10, 3, 5, 1};
        std::make_heap(v.begin(), v.end());           // O(n) build, max-heap by default
    }
}

```

```

assert(v.front() == 10);
v.push_back(42);
std::push_heap(v.begin(), v.end());           // sift the new back() up
assert(v.front() == 42);
std::pop_heap(v.begin(), v.end());             // moves max to back(), re-heaps [begin, end-1)
assert(v.back() == 42);
v.pop_back();
assert(v.front() == 10);
// min-heap: pass std::greater<> to every call
std::make_heap(v.begin(), v.end(), std::greater<>());
assert(v.front() == 1);
}

// ---- std::priority_queue: max by default, min via std::greater, custom via lambda ----
{
    std::priority_queue<int> maxq;
    for (int x : {3, 1, 4, 1, 5}) maxq.push(x);
    assert(maxq.top() == 5);

    std::priority_queue<int, std::vector<int>, std::greater<int>> minq;
    for (int x : {3, 1, 4, 1, 5}) minq.push(x);
    assert(minq.top() == 1);

    // pairs compare lexicographically: (dist, id) - smallest dist on top
    using P = std::pair<double, int>;
    std::priority_queue<P, std::vector<P>, std::greater<P>> pq;
    pq.push({2.5, 7}); pq.push({0.5, 3}); pq.push({1.0, 9});
    assert(pq.top().second == 3);

    // custom comparator with a lambda: "a comes AFTER b" == a has lower priority
    struct Task { int prio; std::string name; };
    auto cmp = [](const Task& a, const Task& b) { return a.prio < b.prio; }; // max-heap on prio
    std::priority_queue<Task, std::vector<Task>, decltype(cmp)> tasks(cmp);
    tasks.push({2, "plan"}); tasks.push({9, "estop"}); tasks.push({5, "localise"});
    assert(tasks.top().name == "estop");
}

// ---- nth_element: O(n) average selection ----
{
    std::vector<int> v = {9, 2, 7, 4, 5, 1, 8, 3, 6};
    int k = 3; // 3rd smallest (0-based index 2)
    std::nth_element(v.begin(), v.begin() + (k - 1), v.end());
    assert(v[k - 1] == 3);
    // everything left of the pivot is <= it, right is >= it, but NOT sorted
    for (int i = 0; i < k - 1; ++i) assert(v[i] <= v[k - 1]);
    for (int i = k; i < static_cast<int>(v.size()); ++i) assert(v[i] >= v[k - 1]);
}

// ---- TreeNode build + traversals ----
{
    // tree: 3 -> (9, 20), 20 -> (15, 7)
    TreeNode* root = buildTree({3, 9, 20, NONE, NONE, 15, 7});
    std::vector<int> in, pre;
    inorder(root, in);
    preorder(root, pre);
    assert(in == std::vector<int>({9, 3, 15, 20, 7}));
    assert(pre == std::vector<int>({3, 9, 20, 15, 7}));
    assert(inorderIterative(root) == in);
    deleteTree(root);
    assert(buildTree({}) == nullptr);
}

// ---- std::list::splice + iterator stability (the LRU building block) ----
{
    std::list<int> l = {1, 2, 3, 4};
    std::unordered_map<int, std::list<int>::iterator> where;
    for (auto it = l.begin(); it != l.end(); ++it) where[*it] = it;
    // move node "3" to the front in O(1): no allocation, no copy, iterators stay valid
    l.splice(l.begin(), l, where[3]);
    assert(l == std::list<int>({3, 1, 2, 4}));
}

```

```

    assert(*where[3] == 3 && where[3] == l.begin()); // the stored iterator still points at 3
    l.erase(where[2]);
    where.erase(2);
    assert(l == std::list<int>({3, 1, 4}));
}

std::cout << "OK 00_heap_tree_toolkit.cpp\n";
return 0;
}

```

What to look at:

- `MaxHeap::siftUp` and `MaxHeap::siftDown`: the two loops from A1. Match each line to the trace of push 45 and pop.
- `std::make_heap`, `std::push_heap`, `std::pop_heap`: the same heap as `priority_queue` but on a vector you can still look inside. `pop_heap` moves the biggest to the *back* of the vector and leaves it to you to `pop_back()`.
- The three `priority_queue` declarations in `main`: biggest on top, smallest on top, and a custom comparison with a lambda.
- `nth_element` and the two loops after it that check "everything before is smaller or equal, everything after is bigger or equal".
- `buildTree` for `{3, 9, 20, NONE, NONE, 15, 7}` and the asserted in-order and pre-order results. Draw the tree and check them.
- The `splice` block: after the move, `where[3]` still points at the right node.

Compile and run:

```

cd code
g++ -std=c++17 -Wall -Wextra -O2 05-heaps-trees-design/00_heap_tree_toolkit.cpp -o toolkit && ./toolkit

```

Part B — The problems

Same shape as chapter 03: the full problem with a drawing, how to do it by hand, the Python, the C++, a walk-through with real values, then the follow-ups. Read the statement, close the file, and try it in C++ for 25 minutes before reading the rest.

1. Kth Largest Element in an Array (LeetCode 215, medium)

The problem in plain words. You are given a list of whole numbers (not sorted, possibly with repeats) and a number k . Return the k th largest number. "kth largest" counts repeats: in `[5, 5, 4]` the 2nd largest is 5, not 4.

```

nums = [3, 2, 1, 5, 6, 4], k = 2

sorted biggest first:  6  5  4  3  2  1
                     ^  ^
                   1st 2nd          -> answer 5

```

k is always between 1 and the length of the list. The list has up to 100,000 numbers, and $k = 1$ means "the maximum", $k = \text{length}$ means "the minimum".

By hand. For a short list you sort it and count k from the big end. For a long list you would rather keep a short list of the k biggest seen so far, and for each new number ask "is it bigger than the smallest one on my short list?" If yes, it replaces that one. At the end the smallest on the short list is the answer. That is the VIP list from A2.

The idea. Two ways, both in the file.

1. A min-heap of size k (A2). Push every number; whenever the heap has more than k numbers, pop the smallest. At the end the top is the k th largest.
2. `std::nth_element` (A2). The k th largest is the element at position $n - k$ in ascending order. Ask `nth_element` to put the right element at that position.

The Python you would have written.

```
import heapq

def findKthLargest(nums, k):
    h = []
    for x in nums:
        heapq.heappush(h, x)
        if len(h) > k:
            heapq.heappop(h)
    return h[0]
```

In C++.

```
// LeetCode 215. Kth Largest Element in an Array
#include <algorithm>
#include <cassert>
#include <functional>
#include <iostream>
#include <queue>
#include <vector>

// Approach 1: min-heap of size k. The heap holds the k largest seen so far;
// its top is the smallest of those, i.e. the answer. O(n log k) time, O(k) space.
int findKthLargest(std::vector<int>& nums, int k) {
    std::priority_queue<int, std::vector<int>, std::greater<int>> minHeap;
    for (int x : nums) {
        minHeap.push(x);
        if (static_cast<int>(minHeap.size()) > k) minHeap.pop(); // evict the smallest
    }
    return minHeap.top();
}

// Approach 2: quickselect via std::nth_element. O(n) average, O(n^2) worst
// (libstdc++ uses introselect so in practice it never degrades). Reorders nums.
int findKthLargestSelect(std::vector<int>& nums, int k) {
    // kth largest == element at index n-k once sorted ascending
    auto target = nums.begin() + (static_cast<int>(nums.size()) - k);
    std::nth_element(nums.begin(), target, nums.end());
    return *target;
}

int main() {
    auto check = [](std::vector<int> nums, int k, int expected) {
        std::vector<int> copy = nums; // heap version does not modify, select does
        assert(findKthLargest(nums, k) == expected);
        assert(findKthLargestSelect(copy, k) == expected);
    };
    check({3, 2, 1, 5, 6, 4}, 2, 5);
    check({3, 2, 3, 1, 2, 4, 5, 5, 6}, 4, 4);
    check({1}, 1, 1); // single element
    check({2, 1}, 2, 1); // k == n -> the minimum
    check({-1, -1, -1}, 2, -1); // duplicates and negatives
    std::cout << "OK 01_kth_largest.cpp\n";
    return 0;
}
```

What is new:

- `std::priority_queue<int, std::vector<int>, std::greater<int>> minHeap;` is the "smallest on top" heap from A1; it replaces `h = []` plus the `heapq` calls.
- `static_cast<int>(minHeap.size()) > k` is `len(h) > k`. `size()` gives an unsigned number; converting it to `int` before comparing with the `int` `k` keeps the compiler quiet.
- `auto target = nums.begin() + (n - k);` is a position (an iterator) inside the vector, "the slot at index `n - k`". `*target` reads the value at that position, like `nums[n - k]`.
- `std::nth_element(nums.begin(), target, nums.end());` reorders `nums` so that `*target` is the element a full sort would put there. This replaces `sorted(nums)[n - k]` but is linear on average.
- In `main`, check is a *lambda*: a small function written inline, like a Python `def` inside `main`. It copies `nums` so the `nth_element` version can reorder it without affecting the heap version.

Walk through the example. `nums = [3, 2, 1, 5, 6, 4]`, `k = 2`. Heap shown with its top first.

```
3: push -> {3}
2: push -> {2, 3}
1: push -> {1, 2, 3}    size 3 > 2, pop 1    -> {2, 3}
5: push -> {2, 3, 5}    pop 2              -> {3, 5}
6: push -> {3, 5, 6}    pop 3              -> {5, 6}
4: push -> {4, 5, 6}    pop 4              -> {5, 6}
top = 5
```

With `nth_element`: `n = 6`, `k = 2`, so `target` is index 4. Ascending order is `[1, 2, 3, 4, 5, 6]` and index 4 holds 5. After the call `nums[4] == 5`, the four numbers before it are all `<= 5` and the one after is `>= 5`, in some order.

Complexity. Heap: one pass, each step costs `log k`: `O(n log k)` time, `O(k)` extra memory. `nth_element`: linear on average, `O(n)`, no extra memory, but it reorders the input.

Robotics. "The 95th-percentile range in this lidar scan" or "the `k`th smallest error in a batch of matches" is exactly this. If the batch is already in a vector, `nth_element` is the tool; if readings stream in from the sensor, the size-`k` heap is.

Follow-ups you may get. - *The numbers arrive one at a time and you cannot store them all?* The heap of size `k`; it only ever holds `k` numbers. - *Worst case of quickselect?* Quadratic with unlucky pivots; the standard library's `nth_element` mixes in a fallback so in practice it stays linear. - *Values are small whole numbers in a known range?* Count how many of each (counting sort), then walk down from the top until you have passed `k` of them.

2. K Closest Points to Origin (LeetCode 973, medium)

The problem in plain words. You are given a list of points on a flat plane, each an `[x, y]` pair of whole numbers, and a number `k`. Return the `k` points that are closest to the origin `(0, 0)`, measured by straight-line distance. The answer may be in any order.

```
points = [[3, 3], [5, -1], [-2, 4]], k = 2
```

```

y
4 |      *(-2,4)
3 |      *(3,3)
  |
0 +-----> x
-1 |      *(5,-1)
```

```
distance squared from (0,0):  (3,3) -> 9+9 = 18    (5,-1) -> 25+1 = 26    (-2,4) -> 4+16 = 20
two closest: (3,3) and (-2,4)                                -> answer [[3,3], [-2,4]]
```

Up to 10,000 points, coordinates between -10,000 and 10,000, `k` between 1 and the number of points.

By hand. Work out each point's distance from the origin, then pick the k smallest. One saving: you never need the actual distance, only which is smaller. The distance is $\sqrt{x^2 + y^2}$, and a bigger $x^2 + y^2$ always means a bigger square root, so compare $x^2 + y^2$ and skip the square root.

The idea. The VIP list from A2 again, but now we want the k *smallest* distances, so the heap keeps the *biggest* on top: the worst VIP is the farthest point, and a newcomer gets in only if it is closer than that. Push (distance squared, index of the point) pairs; when the heap exceeds size k , pop the top (the farthest). At the end, the heap holds the k closest.

The Python you would have written.

```
import heapq

def kClosest(points, k):
    h = []                                # max-heap via negative distances
    for i, (x, y) in enumerate(points):
        heapq.heappush(h, (-(x * x + y * y), i))
        if len(h) > k:
            heapq.heappop(h)              # drops the farthest
    return [points[i] for _, i in h]
```

In C++.

```
// LeetCode 973. K Closest Points to Origin
#include <algorithm>
#include <cassert>
#include <iostream>
#include <queue>
#include <vector>

// Max-heap of size k keyed on squared distance: the top is the FARTHEST of the
// k best, so a new point only enters if it beats that. sqrt is monotonic, so
// comparing squared distances gives the same order without the float call.
// O(n log k) time, O(k) space.
std::vector<std::vector<int>>> kClosest(std::vector<std::vector<int>>& points, int k) {
    using Entry = std::pair<long long, int>; // (squared distance, index into points)
    std::priority_queue<Entry> maxHeap;      // default std::less -> largest distance on top
    for (int i = 0; i < static_cast<int>(points.size()); ++i) {
        long long x = points[i][0], y = points[i][1];
        maxHeap.push({x * x + y * y, i});
        if (static_cast<int>(maxHeap.size()) > k) maxHeap.pop();
    }
    std::vector<std::vector<int>>> out;
    out.reserve(k);
    while (!maxHeap.empty()) {
        out.push_back(points[maxHeap.top().second]);
        maxHeap.pop();
    }
    return out;
}

// Alternative: nth_element with a distance comparator. O(n) average, reorders points.
std::vector<std::vector<int>>> kClosestSelect(std::vector<std::vector<int>>& points, int k) {
    auto dist2 = [](const std::vector<int>& p) {
        return static_cast<long long>(p[0] * p[0] + static_cast<long long>(p[1]) * p[1]);
    };
    std::nth_element(points.begin(), points.begin() + k, points.end(),
        [&](const std::vector<int>& a, const std::vector<int>& b) { return dist2(a) < dist2(b); });
    return std::vector<std::vector<int>>>(points.begin(), points.begin() + k);
}

// Order of the answer is unspecified, so compare as sorted sets.
static std::vector<std::vector<int>>> sorted(std::vector<std::vector<int>>> v) {
    std::sort(v.begin(), v.end());
    return v;
}

int main() {
    {
        std::vector<std::vector<int>>> pts = {{1, 3}, {-2, 2}};
        auto copy = pts;
        assert(sorted(kClosest(pts, 1)) == sorted({{-2, 2}}));
    }
}
```

```

    assert(sorted(kClosestSelect(copy, 1)) == sorted({{-2, 2}}));
}
{
    std::vector<std::vector<int>> pts = {{3, 3}, {5, -1}, {-2, 4}};
    auto copy = pts;
    assert(sorted(kClosest(pts, 2)) == sorted({{3, 3}, {-2, 4}}));
    assert(sorted(kClosestSelect(copy, 2)) == sorted({{3, 3}, {-2, 4}}));
}
{
    std::vector<std::vector<int>> pts = {{0, 1}, {1, 0}};    // k == n, returns everything
    assert(sorted(kClosest(pts, 2)) == sorted({{0, 1}, {1, 0}}));
}
{
    std::vector<std::vector<int>> pts = {{10000, 10000}};    // squares need 64-bit headroom
    assert(kClosest(pts, 1).size() == 1);
}
std::cout << "OK 02_k_closest_points.cpp\n";
return 0;
}

```

What is new:

- `using Entry = std::pair<long long, int>;` names the pair type (distance squared, index). `long long` is a 64-bit whole number; `10000 * 10000 + 10000 * 10000` fits in a plain `int`, but widening is the habit that saves you when the limits change.
- `std::priority_queue<Entry> maxHeap;` with nothing else in the brackets is "biggest on top", and pairs compare by their first element, the distance. There is no need for the Python minus-sign trick.
- `long long x = points[i][0], y = points[i][1];` reads the two coordinates into 64-bit numbers *before* multiplying, so the multiplication itself is done in 64 bits.
- `out.reserve(k);` tells the vector to make room for `k` items up front. Optional; it avoids a few reallocations.
- `while (!maxHeap.empty()) { out.push_back(points[maxHeap.top().second]); maxHeap.pop(); }` is the Python list comprehension: `.second` is the stored index, and `points[...]` is the point itself.
- The second function, `kClosestSelect`, does the same with `std::nth_element` and a comparison lambda `[&](a, b) { return dist2(a) < dist2(b); }` that says "a comes before b if it is closer". The `[&]` lets the lambda use `dist2` from the enclosing function.

Walk through the example. `points = [[3,3], [5,-1], [-2,4]]`, `k = 2`. Heap shown top first.

```

i=0, (3,3):    dist2 = 18. push (18,0)          -> {(18,0)}
i=1, (5,-1):   dist2 = 26. push (26,1)          -> {(26,1), (18,0)}      size 2, ok
i=2, (-2,4):   dist2 = 20. push (20,2)          -> {(26,1), (20,2), (18,0)}
                size 3 > 2: pop top (26,1)      -> {(20,2), (18,0)}
drain: pop (20,2) -> points[2] = (-2,4);  pop (18,0) -> points[0] = (3,3)
out = [[-2,4], [3,3]]

```

The test compares sorted copies because any order is accepted.

Complexity. One pass, $\log k$ per point: $O(n \log k)$ time, $O(k)$ memory. The `nth_element` version is $O(n)$ average and reorders the input.

Robotics. This is k -nearest-neighbour search on a point cloud with no index at all. Real libraries add a KD-tree so that most points are never even looked at, but the "keep a bounded max-heap of the k best while searching" part inside them is exactly this loop, and they compare squared distances for the same reason.

Follow-ups you may get. - *k is close to n?* The heap approaches $n \log n$; just sort by distance or use `nth_element`. - *Points keep arriving and you must always know the current k closest?* Keep the same heap alive between calls; it is already a streaming structure. - *3D and many queries from different centres?* Build a KD-tree once, then each query is about $\log n$.

3. Kth Smallest in a Sorted Matrix (LeetCode 378, medium)

The problem in plain words. You are given an n by n grid of whole numbers where every row is sorted left to right and every column is sorted top to bottom. Return the k th smallest number in the whole grid, counting repeats.

```
matrix =   1   5   9
          10  11  13
          12  13  15          k = 8

all values in order: 1, 5, 9, 10, 11, 12, 13, 13, 15
                    1 2 3  4  5  6  7  8  9      -> the 8th is 13
```

n is at most 300, values between -1,000,000,000 and 1,000,000,000, and k between 1 and $n*n$.

By hand. Because every row is sorted, the smallest value overall must be at the *front* of some row. Take it, cross it off, and now the next smallest is again at the front of some row (possibly the same row, one step along). Repeat k times. You only ever compare the n row fronts, never the whole grid.

The idea. The "merge n sorted lists" pattern. Put the first cell of every row into a min-heap as `(value, row, col)`. Pop the smallest; that is the next value in order. Then push the next cell in that same row, if there is one. The k th pop is the answer.

The file also has a second method that uses no heap: guess a value, count how many cells are $\leq$ the guess by walking a staircase from the bottom-left corner, and binary-search the guess. Read it after the heap version is solid.

The Python you would have written.

```
import heapq

def kthSmallest(matrix, k):
    n = len(matrix)
    h = [(matrix[r][0], r, 0) for r in range(min(n, k))]
    heapq.heapify(h)
    while True:
        val, r, c = heapq.heappop(h)
        k -= 1
        if k == 0:
            return val
        if c + 1 < n:
            heapq.heappush(h, (matrix[r][c + 1], r, c + 1))
```

In C++.

```
// LeetCode 378. Kth Smallest Element in a Sorted Matrix
#include <algorithm>
#include <cassert>
#include <functional>
#include <iostream>
#include <queue>
#include <tuple>
#include <vector>

// Approach 1: n-way merge with a min-heap. Seed the heap with the head of each
// row (rows are sorted), pop k-1 times, each pop pushes that row's next element.
// O(k log n) time, O(n) space where n = number of rows (capped at k).
int kthSmallest(std::vector<std::vector<int>>& matrix, int k) {
    int n = static_cast<int>(matrix.size());
    using Cell = std::tuple<int, int, int>; // (value, row, col) - tuple compares lexicographically
    std::priority_queue<Cell, std::vector<Cell>, std::greater<Cell>> minHeap;
    for (int r = 0; r < std::min(n, k); ++r) minHeap.push({matrix[r][0], r, 0});
    while (true) {
        auto [val, r, c] = minHeap.top();
        minHeap.pop();
        if (--k == 0) return val;
        if (c + 1 < n) minHeap.push({matrix[r][c + 1], r, c + 1});
    }
}
```

```

}

// Approach 2: binary search on the VALUE range, not on indices. For a candidate
// mid, count how many cells are <= mid in O(n) by walking from the bottom-left
// corner (a "staircase" walk). Smallest mid with count >= k is the answer.
// O(n log(max-min)) time, O(1) space. No heap at all.
int kthSmallestBinarySearch(std::vector<std::vector<int>>& matrix, int k) {
    int n = static_cast<int>(matrix.size());
    auto countLessEqual = [&](int target) {
        int count = 0;
        int r = n - 1, c = 0;
        while (r >= 0 && c < n) {
            if (matrix[r][c] <= target) { count += r + 1; ++c; } // whole column above r is <= target
            else --r;
        }
        return count;
    };
    int lo = matrix[0][0], hi = matrix[n - 1][n - 1];
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (countLessEqual(mid) >= k) hi = mid; // answer is <= mid
        else lo = mid + 1;
    }
    return lo; // lo is always an actual matrix value: it's the smallest value with count >= k
}

int main() {
    {
        std::vector<std::vector<int>> m = {{1, 5, 9}, {10, 11, 13}, {12, 13, 15}};
        assert(kthSmallest(m, 8) == 13);
        assert(kthSmallestBinarySearch(m, 8) == 13);
        assert(kthSmallest(m, 1) == 1);
        assert(kthSmallestBinarySearch(m, 1) == 1);
        assert(kthSmallest(m, 9) == 15);
        assert(kthSmallestBinarySearch(m, 9) == 15);
    }
    {
        std::vector<std::vector<int>> m = {{-5}}; // 1x1
        assert(kthSmallest(m, 1) == -5);
        assert(kthSmallestBinarySearch(m, 1) == -5);
    }
    {
        std::vector<std::vector<int>> m = {{1, 2}, {1, 3}}; // duplicates across rows
        assert(kthSmallest(m, 2) == 1);
        assert(kthSmallestBinarySearch(m, 2) == 1);
        assert(kthSmallest(m, 3) == 2);
        assert(kthSmallestBinarySearch(m, 3) == 2);
    }
    std::cout << "OK 03_kth_smallest_sorted_matrix.cpp\n";
    return 0;
}

```

What is new:

- using `Cell = std::tuple<int, int, int>`; is a three-item pair, (value, row, col). Tuples compare item by item from the left, so the heap orders by value first. Put the thing you sort by first, as with pairs.
- `std::priority_queue<Cell, std::vector<Cell>, std::greater<Cell>> minHeap`; is the smallest-on-top heap of tuples.
- `for (int r = 0; r < std::min(n, k); ++r)` seeds the heap. If `k` is smaller than `n`, rows beyond the `k`th can never contribute, so we skip them.
- `auto [val, r, c] = minHeap.top(); minHeap.pop();` unpacks the tuple like Python's `val, r, c = heappop(h)`, in two calls (look, then remove).
- `if (--k == 0) return val;` decreases `k` first, then tests it. Same as `k -= 1`; if `k == 0`.
- In the second function, `auto countLessEqual = [&](int target) { ... };` is a lambda that can see `matrix` and `n` from the outer function (`[&]`). `int mid = lo + (hi - lo) / 2;` is the overflow-safe way to write `(lo + hi) / 2` when values can be near a billion.

Walk through the example. `k = 8`. Seed the heap with the row fronts: `{(1,0,0), (10,1,0), (12,2,0)}`. Heap shown with its top first.

```

pop (1, r0,c0)    k=7    push (5, r0,c1)    heap {5, 10, 12}
pop (5, r0,c1)    k=6    push (9, r0,c2)    heap {9, 10, 12}
pop (9, r0,c2)    k=5    row 0 is finished  heap {10, 12}
pop (10, r1,c0)   k=4    push (11, r1,c1)   heap {11, 12}
pop (11, r1,c1)   k=3    push (13, r1,c2)   heap {12, 13}
pop (12, r2,c0)   k=2    push (13, r2,c1)   heap {13(r1), 13(r2)}
pop (13, r1,c2)   k=1    row 1 is finished  heap {13(r2)}
pop (13, r2,c1)   k=0    return 13

```

The binary-search version on the same input: $lo = 1$, $hi = 15$. Guess 8: only 2 cells are ≤ 8 , fewer than 8, so $lo = 9$. Guess 12: 6 cells, $lo = 13$. Guess 14: 8 cells, $hi = 14$. Guess 13: 8 cells, $hi = 13$. Now $lo == hi == 13$. Answer 13.

Complexity. Heap: k pops, each $\log n$ with at most n things in the heap: $O(k \log n)$ time, $O(n)$ memory. Binary search on the value: each count is one staircase walk of at most $2n$ steps, and there are about $\log(\max - \min)$ guesses: $O(n \log(\text{range}))$ time, no extra memory.

Robotics. Merging several time-sorted message streams (one per sensor) into one ordered stream is exactly the n -way merge: the heap holds the next message from each stream. The binary-search-on-value trick is how you find a distance threshold that keeps exactly a chosen fraction of the closest laser returns.

Follow-ups you may get. - *Why not copy everything into one vector and use `nth_element`?* It works but costs nn time and memory and throws away the fact that the rows are sorted. - What if only the rows are sorted, not the columns? *The heap merge still works (it only needs sorted rows). The staircase count does not.* - Why does the binary search land on a value that is actually in the grid? The count only changes at values that exist in the grid, so the smallest guess with count $\geq k$ is one of them.

4. Sliding Window Maximum (LeetCode 239, hard)

The problem in plain words. You are given a list of numbers and a window width k . Slide the window from the left end to the right end one step at a time, and for every position report the largest number inside the window.

```

nums = [1, 3, -1, -3, 5, 3, 6, 7], k = 3

[1 3 -1] -3 5 3 6 7    -> 3
1 [3 -1 -3] 5 3 6 7    -> 3
1 3 [-1 -3 5] 3 6 7    -> 5
1 3 -1 [-3 5 3] 6 7    -> 5
1 3 -1 -3 [5 3 6] 7    -> 6
1 3 -1 -3 5 [3 6 7]    -> 7          answer: [3, 3, 5, 5, 6, 7]

```

The list has up to 100,000 numbers and k is between 1 and the list length. With $k = 1$ the answer is the list itself; with $k = \text{length}$ it is one number, the maximum.

By hand. Slide the window and look at the three numbers each time. With 8 numbers that is fine; with 100,000 numbers and $k = 50,000$ it is far too slow. The fast way is the monotonic deque from A3: keep only the numbers that could still become a maximum.

The idea. For each index i : 1. If the index at the front of the deque is $\leq i - k$, it has slid out of the window: pop it from the front. 2. While the value at the back of the deque is $\leq$ the new value, pop the back (it is older and no bigger, so it can never win). 3. Push i at the back. 4. Once $i \geq k - 1$ (the first full window), the front of the deque is the maximum: record it.

The Python you would have written.

```

from collections import deque

def maxSlidingWindow(nums, k):
    dq = deque()
    out = []
    # indices; values decreasing from front to back

```

```

for i, x in enumerate(nums):
    if dq and dq[0] <= i - k:
        dq.popleft()
    while dq and nums[dq[-1]] <= x:
        dq.pop()
    dq.append(i)
    if i >= k - 1:
        out.append(nums[dq[0]])
return out

```

In C++.

```

// LeetCode 239. Sliding Window Maximum
#include <cassert>
#include <deque>
#include <iostream>
#include <vector>

// Monotonic deque of INDICES, values decreasing from front to back.
// Front is always the max of the current window. Each index is pushed and
// popped at most once -> O(n) total, O(k) space.
std::vector<int> maxSlidingWindow(std::vector<int>& nums, int k) {
    std::deque<int> dq;
    std::vector<int> out;
    int n = static_cast<int>(nums.size());
    for (int i = 0; i < n; ++i) {
        // 1. drop indices that fell out of the window [i-k+1, i]
        if (!dq.empty() && dq.front() <= i - k) dq.pop_front();
        // 2. a new, larger value makes every smaller one behind it useless:
        // they are older AND smaller, so they can never be a window max again
        while (!dq.empty() && nums[dq.back()] <= nums[i]) dq.pop_back();
        dq.push_back(i);
        // 3. once the first full window is in, record its max
        if (i >= k - 1) out.push_back(nums[dq.front()]);
    }
    return out;
}

int main() {
    {
        std::vector<int> nums = {1, 3, -1, -3, 5, 3, 6, 7};
        assert(maxSlidingWindow(nums, 3) == std::vector<int>({3, 3, 5, 5, 6, 7}));
    }
    {
        std::vector<int> nums = {1};
        assert(maxSlidingWindow(nums, 1) == std::vector<int>({1}));
    }
    {
        std::vector<int> nums = {9, 8, 7, 6}; // strictly decreasing: front expires each step
        assert(maxSlidingWindow(nums, 2) == std::vector<int>({9, 8, 7}));
    }
    {
        std::vector<int> nums = {1, 2, 3, 4}; // strictly increasing: deque holds one index
        assert(maxSlidingWindow(nums, 2) == std::vector<int>({2, 3, 4}));
    }
    {
        std::vector<int> nums = {4, 4, 4}; // ties: <= pops the older duplicate, still correct
        assert(maxSlidingWindow(nums, 3) == std::vector<int>({4}));
    }
    std::cout << "OK 04_sliding_window_maximum.cpp\n";
    return 0;
}

```

What is new:

- `#include <deque>` and `std::deque<int> dq;` is `collections.deque`. `push_back`, `pop_back`, `back()`, `front()`, `pop_front()` are `append`, `pop`, `dq[-1]`, `dq[0]`, `popleft`. As always in C++, "look" and "remove" are separate calls.
- `if (!dq.empty() && dq.front() <= i - k)` is `if dq and dq[0] <= i - k`. The `!dq.empty()` must come first; `front()` on an empty deque is undefined behaviour.
- `while (!dq.empty() && nums[dq.back()] <= nums[i]) dq.pop_back();` is the "throw out the shorter people behind" loop from A3.

- `if (i >= k - 1) out.push_back(nums[dq.front()]);` records the answer only once the window is full.

Walk through the example. Deque as `[index(value), ...]`, front on the left.

<code>i=0 (1):</code>	<code>push 0</code>	<code>[0(1)]</code>	
<code>i=1 (3):</code>	<code>back 1 <= 3, pop. push 1</code>	<code>[1(3)]</code>	
<code>i=2 (-1):</code>	<code>back 3 > -1, keep. push 2</code>	<code>[1(3), 2(-1)]</code>	<code>out: 3</code>
<code>i=3 (-3):</code>	<code>front 1 <= 0? no. back -1 > -3. push 3</code>	<code>[1(3), 2(-1), 3(-3)]</code>	<code>out: 3</code>
<code>i=4 (5):</code>	<code>front 1 <= 1? yes, pop front</code>	<code>[2(-1), 3(-3)]</code>	
	<code>back -3 <= 5 pop; back -1 <= 5 pop. push 4</code>	<code>[4(5)]</code>	<code>out: 5</code>
<code>i=5 (3):</code>	<code>front 4 <= 2? no. back 5 > 3. push 5</code>	<code>[4(5), 5(3)]</code>	<code>out: 5</code>
<code>i=6 (6):</code>	<code>front 4 <= 3? no. back 3 <= 6 pop,</code>		
	<code>back 5 <= 6 pop. push 6</code>	<code>[6(6)]</code>	<code>out: 6</code>
<code>i=7 (7):</code>	<code>front 6 <= 4? no. back 6 <= 7 pop. push 7</code>	<code>[7(7)]</code>	<code>out: 7</code>
<code>out = [3, 3, 5, 5, 6, 7]</code>			

Notice at `i=4`: the deque still held `-1` and `-3`, but `5` threw both out at once. They had no chance of ever being a maximum again.

Complexity. Each index is pushed once and popped at most once, so the whole thing is one pass: $O(n)$ time. The deque never holds more than k indices: $O(k)$ memory.

Robotics. "Maximum (or minimum) of the last k sensor readings" is a windowed filter: peak detection on a force sensor, the biggest range in the last k laser scans for an adaptive obstacle threshold, an envelope on a microphone channel. The deque makes it constant work per sample, which is what lets it run inside a 1 kHz control loop.

Follow-ups you may get. - *Sliding window minimum?* The same code with the comparison flipped (`>=` instead of `<=`). - *Could you use a heap?* Yes: a max-heap of `(value, index)`, and when the top's index is too old, pop it and look again. That is $O(n \log k)$; the deque is better. - *Sliding window median?* Two heaps (a max-heap for the lower half, a min-heap for the upper half) or a `std::multiset` with a pointer to the middle, $\log k$ per step.

5. Binary Tree Level Order Traversal (LeetCode 102, medium)

The problem in plain words. You are given the root of a binary tree. Return its values level by level: first a list with the root, then a list with the root's children left to right, then the grandchildren left to right, and so on. The answer is a list of lists.

tree given as `[3, 9, 20, null, null, 15, 7]`:



level 0: `[3]`

level 1: `[9, 20]`

level 2: `[15, 7]`

answer: `[[3], [9, 20], [15, 7]]`

The tree can be empty (answer: an empty list) and can have up to 2,000 nodes. A tree that is one long chain gives one single-item list per level.

By hand. Read the drawing row by row. Each row of the drawing is one level. To do it without a drawing: write the root on a piece of paper; then for each node on the paper, write its children on the next piece of paper; when the first paper is used up, that was one level, and you move to the next paper.

The idea. BFS with a queue and the level-size snapshot from A4 (the ring loop from chapter 03). Push the root. While the queue is not empty: note how many nodes are in the queue right now (that is this level), pop exactly that many, record their values, and push their children (they will be the next level). Append the level's list to the answer.

The Python you would have written.

```

from collections import deque

def levelOrder(root):
    out = []
    if root is None:
        return out
    q = deque([root])
    while q:
        level = []
        for _ in range(len(q)):          # exactly this level
            node = q.popleft()
            level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        out.append(level)
    return out

```

In C++.

```

// LeetCode 102. Binary Tree Level Order Traversal
#include <cassert>
#include <iostream>
#include <queue>
#include <vector>

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode() : val(0), left(nullptr), right(nullptr) {}
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
    TreeNode(int x, TreeNode* left, TreeNode* right) : val(x), left(left), right(right) {}
};

// BFS with a queue. The trick that separates levels: snapshot q.size() at the
// start of each round – that many nodes are exactly the current level, and
// everything pushed during the round belongs to the next one.
// O(n) time, O(w) space where w = widest level.
std::vector<std::vector<int>> levelOrder(TreeNode* root) {
    std::vector<std::vector<int>> out;
    if (root == nullptr) return out;
    std::queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        int levelSize = static_cast<int>(q.size());    // freeze it: q grows inside the loop
        std::vector<int> level;
        level.reserve(levelSize);
        for (int i = 0; i < levelSize; ++i) {
            TreeNode* cur = q.front();
            q.pop();
            level.push_back(cur->val);
            if (cur->left) q.push(cur->left);
            if (cur->right) q.push(cur->right);
        }
        out.push_back(std::move(level));
    }
    return out;
}

// ---- test helpers -----
const int NONE = -1;
TreeNode* buildTree(const std::vector<int>& vals) {
    if (vals.empty() || vals[0] == NONE) return nullptr;
    TreeNode* root = new TreeNode(vals[0]);
    std::queue<TreeNode*> q;
    q.push(root);

```

```

int i = 1, n = static_cast<int>(vals.size());
while (!q.empty() && i < n) {
    TreeNode* cur = q.front();
    q.pop();
    if (i < n && vals[i] != NONE) { cur->left = new TreeNode(vals[i]); q.push(cur->left); }
    ++i;
    if (i < n && vals[i] != NONE) { cur->right = new TreeNode(vals[i]); q.push(cur->right); }
    ++i;
}
return root;
}

void deleteTree(TreeNode* root) {
    if (root == nullptr) return;
    deleteTree(root->left);
    deleteTree(root->right);
    delete root;
}

int main() {
    {
        TreeNode* root = buildTree({3, 9, 20, NONE, NONE, 15, 7});
        assert(levelOrder(root) == std::vector<std::vector<int>>({{3}, {9, 20}, {15, 7}}));
        deleteTree(root);
    }
    {
        TreeNode* root = buildTree({1});
        assert(levelOrder(root) == std::vector<std::vector<int>>({{1}}));
        deleteTree(root);
    }
    {
        TreeNode* root = buildTree({1, NONE, 2, NONE, 3}); // right-leaning chain
        assert(levelOrder(root) == std::vector<std::vector<int>>({{1}, {2}, {3}}));
        deleteTree(root);
    }
    assert(levelOrder(nullptr).empty()); // empty tree
    std::cout << "OK 05_level_order_traversal.cpp\n";
    return 0;
}

```

What is new:

- `struct TreeNode { ... }` at the top is the node from A4; LeetCode supplies it, and the standalone file has to repeat it.
- `std::queue<TreeNode*> q;` holds node *addresses*. `q.push(root)` puts in the root's address, not a copy of the node.
- `int levelSize = static_cast<int>(q.size());` is the snapshot, Python's `range(len(q))`. It has to be taken before the loop because the queue grows inside it.
- `level.reserve(levelSize);` pre-sizes the level's vector. Optional.
- `if (cur->left) q.push(cur->left);` is `if node.left: q.append(node.left)`; a non-null pointer counts as true.
- `out.push_back(std::move(level));` hands the level's vector over to `out` without copying it. Plain `out.push_back(level)` would work too, one copy slower.
- `buildTree` and `deleteTree` at the bottom are the test helpers from A4. `NONE (-1)` stands for `null`.

Walk through the example. Queue shown front first.

start:	q = [3]		
level, size 1:	pop 3, record; push 9, 20	q = [9, 20]	out = [[3]]
level, size 2:	pop 9, record; no children	q = [20]	
	pop 20, record; push 15, 7	q = [15, 7]	out = [[3], [9, 20]]
level, size 2:	pop 15, record; no children	q = [7]	
	pop 7, record; no children	q = []	out = [[3], [9, 20], [15, 7]]
queue empty:	done		

Complexity. Every node is pushed and popped once: $O(n)$ time. The queue holds at most one level at a time, so memory is the widest level, $O(n)$ in the worst case.

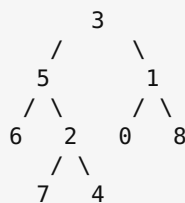
Robotics. Walking a tree level by level is how you print or check the tree hierarchy depth by depth. On an octree it is coarse-to-fine: visit every cell at resolution level d before any at level $d+1$. The frozen-level-size loop is the same one that expands one ring of a wavefront on a grid.

Follow-ups you may get. - *Zigzag order (LeetCode 103)?* Same loop; reverse every other level's list before appending. - *Right side view (LeetCode 199)?* The last value recorded in each level. - *Recursive version?* Pass a depth down: `if depth == out.size(): out.push_back({})`, then `out[depth].push_back(val)`; pre-order keeps each level left to right.

6. Lowest Common Ancestor (LeetCode 236, medium)

The problem in plain words. You are given the root of a binary tree and two nodes p and q that are both in the tree. Return the lowest common ancestor: the deepest node that has both p and q below it, where a node counts as being below itself.

tree [3, 5, 1, 6, 2, 0, 8, null, null, 7, 4]:



$p = 5, q = 1 \rightarrow 3$ (they are on different sides of 3)
 $p = 5, q = 4 \rightarrow 5$ (4 is below 5, and 5 counts as its own ancestor)
 $p = 7, q = 4 \rightarrow 2$
 $p = 6, q = 8 \rightarrow 3$

Between 2 and 100,000 nodes; every value is different; p and q are different nodes that both exist in the tree.

By hand. Put a finger on p and a finger on q . Walk both fingers upwards to their parents, one step each, marking the nodes you pass. The first node both fingers have touched is the answer. That needs parent pointers, which `TreeNode` does not have, so the code uses the "ask both children" method from A5 instead.

The idea. Recursion that returns what it found (A5): 1. If the node is null, or is p , or is q , return the node. 2. Ask the left child and the right child. 3. If both returned something, this node is the LCA: return it. 4. Otherwise return whichever child returned something (or null if neither did).

The Python you would have written.

```
def lowestCommonAncestor(root, p, q):
    if root is None or root is p or root is q:
        return root
    left = lowestCommonAncestor(root.left, p, q)
    right = lowestCommonAncestor(root.right, p, q)
    if left and right:
        return root
    return left if left else right
```

In C++.

```
// LeetCode 236. Lowest Common Ancestor of a Binary Tree
#include <cassert>
#include <iostream>
#include <queue>
#include <vector>
```

```

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode() : val(0), left(nullptr), right(nullptr) {}
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
    TreeNode(int x, TreeNode* left, TreeNode* right) : val(x), left(left), right(right) {}
};

// Post-order "return what you found":
// - a subtree returns nullptr if it contains neither p nor q,
// - returns p or q if it contains exactly one of them (or that node itself),
// - the first node where BOTH sides return non-null is the LCA, and it
//   propagates unchanged up to the root.
// If p is an ancestor of q we return p as soon as we hit it, which is correct
// because the problem guarantees both nodes exist in the tree.
// O(n) time, O(h) recursion depth.
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
    if (root == nullptr || root == p || root == q) return root;
    TreeNode* left = lowestCommonAncestor(root->left, p, q);
    TreeNode* right = lowestCommonAncestor(root->right, p, q);
    if (left && right) return root;           // p and q are on different sides: this is the split
    return left ? left : right;              // pass up whichever side found something
}

// BST variant (LeetCode 235): use the ordering, no recursion needed. O(h).
TreeNode* lowestCommonAncestorBST(TreeNode* root, TreeNode* p, TreeNode* q) {
    while (root) {
        if (p->val < root->val && q->val < root->val) root = root->left;
        else if (p->val > root->val && q->val > root->val) root = root->right;
        else return root;                    // they split here (or one equals root)
    }
    return nullptr;
}

// ---- test helpers -----
const int NONE = -1;
TreeNode* buildTree(const std::vector<int>& vals) {
    if (vals.empty() || vals[0] == NONE) return nullptr;
    TreeNode* root = new TreeNode(vals[0]);
    std::queue<TreeNode*> q;
    q.push(root);
    int i = 1, n = static_cast<int>(vals.size());
    while (!q.empty() && i < n) {
        TreeNode* cur = q.front();
        q.pop();
        if (i < n && vals[i] != NONE) { cur->left = new TreeNode(vals[i]); q.push(cur->left); }
        ++i;
        if (i < n && vals[i] != NONE) { cur->right = new TreeNode(vals[i]); q.push(cur->right); }
        ++i;
    }
    return root;
}

TreeNode* find(TreeNode* root, int val) { // values are unique in the LeetCode tests
    if (root == nullptr || root->val == val) return root;
    TreeNode* l = find(root->left, val);
    return l ? l : find(root->right, val);
}

void deleteTree(TreeNode* root) {
    if (root == nullptr) return;
    deleteTree(root->left);
    deleteTree(root->right);
    delete root;
}

int main() {
    {
        TreeNode* root = buildTree({3, 5, 1, 6, 2, 0, 8, NONE, NONE, 7, 4});
        assert(lowestCommonAncestor(root, find(root, 5), find(root, 1))->val == 3);
        assert(lowestCommonAncestor(root, find(root, 5), find(root, 4))->val == 5); // p is ancestor of q
        assert(lowestCommonAncestor(root, find(root, 7), find(root, 4))->val == 2);
        assert(lowestCommonAncestor(root, find(root, 6), find(root, 8))->val == 3);
        deleteTree(root);
    }
}

```

```

    TreeNode* root = buildTree({1, 2}); // two nodes
    assert(lowestCommonAncestor(root, find(root, 1), find(root, 2))->val == 1);
    deleteTree(root);
}
{
    TreeNode* root = buildTree({6, 2, 8, 0, 4, 7, 9, NONE, NONE, 3, 5}); // a BST (LeetCode 235)
    assert(lowestCommonAncestorBST(root, find(root, 2), find(root, 8))->val == 6);
    assert(lowestCommonAncestorBST(root, find(root, 2), find(root, 4))->val == 2);
    assert(lowestCommonAncestor(root, find(root, 3), find(root, 5))->val == 4); // general version agrees
    deleteTree(root);
}
std::cout << "OK 06_lowest_common_ancestor.cpp\n";
return 0;
}

```

What is new:

- `if (root == nullptr || root == p || root == q) return root;` compares *pointers*: two pointers are equal only when they name the same node. That is Python's `is`. Comparing `root->val == p->val` would be wrong if values could repeat.
- `TreeNode* left = lowestCommonAncestor(root->left, p, q);` stores the child's answer, which is a pointer (possibly `nullptr`).
- `if (left && right) return root;` treats a non-null pointer as true.
- `return left ? left : right;` is Python's `left if left else right`.
- `lowestCommonAncestorBST` is a bonus for the case where the tree is a binary search tree (LeetCode 235): if both values are smaller than the current node go left, if both are bigger go right, otherwise this is the split. No recursion needed.
- `find(root, val)` in the test helpers turns a value into the node pointer, so the tests can say `find(root, 5)` for `p`.

Walk through the example. `p = 7, q = 4`, expected 2. Each line is one call; indentation is depth.

```

call(3): not p or q.
  call(5): not p or q.
    call(6): not p or q. call(null) -> null, call(null) -> null. return null.
    call(2): not p or q.
      call(7): I am p. return 7.
      call(4): I am q. return 4.
    both non-null: return 2.
  left = null, right = 2. return 2.
call(1): not p or q.
  call(0): ... both children null. return null.
  call(8): ... return null.
return null.
left = 2, right = null. return 2.

```

And `p = 5, q = 4`: `call(3)` asks `call(5)`, which returns 5 at once because it *is* `p`, without ever looking at 4. `call(1)` returns null. So 3 returns 5. Correct, because the problem promises `q` is in the tree, and it is not on 1's side, so it must be under 5.

Complexity. Every node is visited once: $O(n)$ time. The recursion is as deep as the tree is tall: $O(\text{height})$ memory.

Robotics. ROS `tf2` finds the transform between two frames by locating their lowest common ancestor in the frame tree and composing the two chains up to it; `lookupTransform` is an LCA query with parent pointers. The same appears in a robot's link tree (URDF) when you need the pose of one link relative to another.

Follow-ups you may get. - *`p` or `q` might not be in the tree (LeetCode 1644)?* Count how many of the two you actually met during the walk; return the candidate only if the count is 2. - *Nodes have parent pointers (LeetCode 1650)?* Walk up from both, first equalising depths, or collect `p`'s ancestors in a set and walk up from `q` until you hit one. - *Many LCA queries on one fixed tree?* Precompute " 2^j -th ancestor" tables (binary lifting): $n \log n$ setup, $\log n$ per query.

7. LRU Cache (LeetCode 146, medium)

The problem in plain words. Build a class `LRUCache` with a fixed capacity and two operations:

- `get(key)`: if the key is in the cache, return its value and mark it as just used; otherwise return -1.
- `put(key, value)`: store the value under the key and mark it as just used. If the key was already there, just update its value. If it was new and the cache was already full, first throw out the entry that was used longest ago.

Both operations must take a constant number of steps, however big the cache is.

capacity 2	shelf, front = most recently used	
<code>put(1, 1)</code>	<code>[(1,1)]</code>	
<code>put(2, 2)</code>	<code>[(2,2) (1,1)]</code>	
<code>get(1) -> 1</code>	<code>[(1,1) (2,2)]</code>	1 moves to the front
<code>put(3, 3)</code>	<code>[(3,3) (1,1)]</code>	full: (2,2) at the back is thrown out
<code>get(2) -> -1</code>		2 is gone
<code>put(4, 4)</code>	<code>[(4,4) (3,3)]</code>	(1,1) thrown out
<code>get(1) -> -1</code>		
<code>get(3) -> 3</code>	<code>[(3,3) (4,4)]</code>	
<code>get(4) -> 4</code>	<code>[(4,4) (3,3)]</code>	

Capacity is at least 1, keys and values are non-negative whole numbers, and there can be up to 200,000 operations.

By hand. Play the bookshelf game from A6 on paper, as in the table above. Every `get` or `put` moves that entry to the front; a `put` of a new key on a full shelf first removes the back entry.

The idea. A `std::list` of `(key, value)` pairs as the shelf, front = most recently used, and a `std::unordered_map` from key to a bookmark (iterator) into the list.

- `get`: look the key up in the map. Missing: return -1. Found: splice its node to the front and return the value.
- `put`: look the key up. Found: change the value in the node, splice to the front. New: if the list is at capacity, read the key of the back node, erase that key from the map, pop the back node. Then push the new pair at the front and store its bookmark in the map.

The Python you would have written.

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.d = OrderedDict()           # front = least recent, end = most recent

    def get(self, key):
        if key not in self.d:
            return -1
        self.d.move_to_end(key)
        return self.d[key]

    def put(self, key, value):
        if key in self.d:
            self.d.move_to_end(key)
        elif len(self.d) == self.cap:
            self.d.popitem(last=False)    # drop the least recent
        self.d[key] = value
```

(Python's `OrderedDict` is a hash map plus a doubly linked list underneath; in C++ you wire the two together yourself.)

In C++.

```

// LeetCode 146. LRU Cache
#include <cassert>
#include <iostream>
#include <list>
#include <unordered_map>
#include <utility>

// Recency order lives in a doubly linked list (front = most recently used).
// The hash map gives O(1) key -> list node. std::list iterators stay valid
// through splice/erase of OTHER nodes, which is exactly what makes this O(1);
// a std::vector would invalidate them on every move.
class LRUCache {
public:
    explicit LRUCache(int capacity) : capacity_(capacity) {}

    int get(int key) {
        auto it = map_.find(key);
        if (it == map_.end()) return -1;
        touch(it->second); // move to front, O(1)
        return it->second->second;
    }

    void put(int key, int value) {
        auto it = map_.find(key);
        if (it != map_.end()) {
            it->second->second = value; // update in place, then mark as recent
            touch(it->second);
            return;
        }
        if (static_cast<int>(items_.size()) == capacity_) {
            map_.erase(items_.back().first); // evict least recently used (back)
            items_.pop_back();
        }
        items_.emplace_front(key, value);
        map_[key] = items_.begin();
    }

private:
    using Node = std::pair<int, int>; // (key, value): key needed to erase from map on eviction
    using Iter = std::list<Node>::iterator;

    void touch(Iter it) {
        // relink the node at the front without copying or reallocating; `it` stays valid
        items_.splice(items_.begin(), items_, it);
    }

    int capacity_;
    std::list<Node> items_;
    std::unordered_map<int, Iter> map_;
};

int main() {
    {
        LRUCache c(2); // LeetCode example
        c.put(1, 1);
        c.put(2, 2);
        assert(c.get(1) == 1);
        c.put(3, 3); // evicts key 2 (1 was just touched)
        assert(c.get(2) == -1);
        c.put(4, 4); // evicts key 1
        assert(c.get(1) == -1);
        assert(c.get(3) == 3);
        assert(c.get(4) == 4);
    }
    {
        LRUCache c(1); // capacity 1: every put of a new key evicts
        c.put(5, 50);
        assert(c.get(5) == 50);
        c.put(6, 60);
        assert(c.get(5) == -1);
        assert(c.get(6) == 60);
        c.put(6, 61); // overwrite existing key, no eviction
        assert(c.get(6) == 61);
    }
}

```

```

}
{
    LRUCache c(2);
    c.put(2, 1);
    c.put(2, 2);           // update keeps size at 1
    assert(c.get(2) == 2);
    c.put(1, 1);
    c.put(4, 1);           // evicts 2 (1 is more recent)
    assert(c.get(2) == -1);
    assert(c.get(1) == 1);
}
{
    LRUCache c(3);
    assert(c.get(42) == -1); // get on empty cache
}
std::cout << "OK 07_lru_cache.cpp\n";
return 0;
}

```

What is new:

- `class LRUCache { public: ... private: ... };` is a Python class with the parts labelled. `public:` are the methods anyone can call; `private:` are the helpers and the data. The trailing `_` on `capacity_`, `items_`, `map_` is just a naming habit for member variables.
- `explicit LRUCache(int capacity) : capacity_(capacity) {}` is `__init__`. `explicit` stops C++ from silently turning a plain `int` into an `LRUCache`; always write it on one-argument constructors.
- `using Node = std::pair<int, int>;` and `using Iter = std::list<Node>::iterator;` name the two types so the rest reads easily. `Iter` is the bookmark type from A6.
- `std::list<Node> items_;` is the shelf; `std::unordered_map<int, Iter> map_;` is the key-to-bookmark map.
- `auto it = map_.find(key); if (it == map_.end()) return -1;` is if key not in self.d. `find` returns a map bookmark, and `end()` means "not found". `it->second` is the value stored under the key, which here is the list bookmark; so `it->second->second` is "the list node's value" (the pair's second half) and `it->second->first` would be its key.
- `items_.splice(items_.begin(), items_, it);` in touch is `move_to_end`, but to the front.
- `map_.erase(items_.back().first); items_.pop_back();` is `popitem(last=False)`: read the back node's key, erase it from the map, then drop the node.
- `items_.emplace_front(key, value); map_[key] = items_.begin();` builds the new node at the front, then stores the bookmark of the new front. The order matters: `begin()` must be read *after* the insert.

Walk through the example. Capacity 2. The list is drawn front first, and the map lists the keys that have bookmarks.

<code>put(1,1):</code>	not in map. size 0 < 2. <code>emplace_front (1,1)</code> . map {1}.	list [(1,1)]
<code>put(2,2):</code>	not in map. size 1 < 2. <code>emplace_front (2,2)</code> . map {1,2}.	list [(2,2) (1,1)]
<code>get(1):</code>	in map. <code>splice</code> its node to the front. return 1.	list [(1,1) (2,2)]
<code>put(3,3):</code>	not in map. size 2 == 2: back is (2,2): <code>map.erase(2)</code> , <code>pop_back</code> . <code>emplace_front (3,3)</code> . map {1,3}.	list [(3,3) (1,1)]
<code>get(2):</code>	not in map. return -1.	
<code>put(4,4):</code>	not in map. full: back is (1,1): <code>erase 1</code> , <code>pop_back</code> . <code>emplace_front (4,4)</code> . map {3,4}.	list [(4,4) (3,3)]
<code>get(1):</code>	not in map. return -1.	
<code>get(3):</code>	in map. <code>splice</code> to front. return 3.	list [(3,3) (4,4)]
<code>get(4):</code>	in map. <code>splice</code> to front. return 4.	list [(4,4) (3,3)]

Complexity. Every operation is a map lookup plus a few pointer rewirings: $O(1)$ per operation. Memory is $O(\text{capacity})$.

Robotics. A localiser keeps the last N lidar submaps resident and evicts by recency; a map renderer does the same with costmap tiles and textures. Both are this class with a bigger value type. The same `list + unordered_map` pair is what you would write for an object pool that recycles the least recently used slot.

Follow-ups you may get. - *Why not a `std::vector` for the shelf?* Moving a middle item to the front shifts everything (linear), and growing a vector can move every element in memory, so stored bookmarks become garbage. `std::list` has neither problem. - *Thread safety?* Wrap both operations in a `std::mutex`. A reader-writer lock does not help, because even `get` changes the recency order. - *LFU instead (evict the least frequently used, LeetCode 460)?* One list per frequency count plus a `minFreq` counter; each access splices the node from list `f` to list `f+1`. Still constant time, about three times the bookkeeping.

8. Trapping Rain Water (LeetCode 42, hard)

The problem in plain words. You are given a list of bar heights, each bar one unit wide, standing side by side. It rains. Water collects in the dips between taller bars and runs off the two ends. Return how many unit squares of water are trapped.

```
height = [4, 2, 0, 3, 2, 5]          # is a bar, ~ is trapped water

5 |                                     #
4 | # ~ ~ ~ ~ #
3 | # ~ ~ # ~ #
2 | # # ~ # # #
1 | # # ~ # # #
+-----+
  0 1 2 3 4 5
  4 2 0 3 2 5    <- height

count the ~ squares: 4 + 3 + 1 + 1 = 9    -> answer 9
```

A second example, the classic one:

```
height = [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]

3 |                                     #
2 |           # ~ ~ ~ # # ~ #
1 |   # ~ # # ~ # # # # # #
+-----+
  0 1 2 3 4 5 6 7 8 9 10 11
  0 1 0 2 1 0 1 3 2 1 2 1    <- height

answer 6
```

The list can be empty or have one bar (answer 0), and can have up to 20,000 bars of height up to 100,000. If the bars only go up, or only go down, nothing is trapped.

By hand. Look at one column at a time. Water sits on top of bar `i` up to the level of the *lower* of two walls: the tallest bar anywhere to its left and the tallest bar anywhere to its right. So the water above bar `i` is $\min(\text{tallest on the left}, \text{tallest on the right}) - \text{height}[i]$, and never less than 0. In the first drawing, above bar 2 (height 0): tallest left is 4, tallest right is 5, min is 4, water is $4 - 0 = 4$. Above bar 3 (height 3): $\min(4, 5) - 3 = 1$. Add them all up.

The idea. Three versions, all in the file.

1. *Two arrays* (the obvious one): `leftMax[i]` = tallest bar from 0 to `i`, computed left to right; `rightMax[i]` = tallest from `i` to the end, computed right to left. Then sum $\min(\text{leftMax}[i], \text{rightMax}[i]) - \text{height}[i]$. Linear time, linear extra memory.
2. *Two pointers* (the one the file calls `trap`): the same formula without the arrays. Keep `l` at the left end and `r` at the right end, with running `leftMax` and `rightMax`. Look at the two bars under the pointers. If $\text{height}[l] < \text{height}[r]$, the tallest bar on the right is at least `height[r]`, which is taller than `height[l]`, so the right wall is not the limit: bar `l`'s water level is decided by `leftMax` alone. Settle it, `++l`. Otherwise settle bar `r` with `rightMax`, `--r`. Stop when they meet. Linear time, no extra memory.

3. *Monotonic stack (A3)*: keep a stack of bar indices with decreasing heights. When a taller bar arrives, each bar popped is the floor of a horizontal strip of water bounded by the new bar and the next bar left on the stack. This fills the water layer by layer instead of column by column.

The Python you would have written. The two-pointer version.

```
def trap(height):
    l, r = 0, len(height) - 1
    left_max = right_max = water = 0
    while l < r:
        if height[l] < height[r]:
            left_max = max(left_max, height[l])
            water += left_max - height[l]
            l += 1
        else:
            right_max = max(right_max, height[r])
            water += right_max - height[r]
            r -= 1
    return water
```

In C++.

```
// LeetCode 42. Trapping Rain Water
#include <algorithm>
#include <cassert>
#include <iostream>
#include <vector>

// Water above bar i = min(maxLeft(i), maxRight(i)) - height[i].
// Two pointers: whichever side has the lower running max is the side whose
// water level is already decided (the other side has something at least as
// tall to hold it), so we can settle that bar and move inwards.
// O(n) time, O(1) space.
int trap(std::vector<int>& height) {
    int l = 0, r = static_cast<int>(height.size()) - 1;
    int leftMax = 0, rightMax = 0, water = 0;
    while (l < r) {
        if (height[l] < height[r]) {
            leftMax = std::max(leftMax, height[l]);
            water += leftMax - height[l];           // never negative: leftMax >= height[l]
            ++l;
        } else {
            rightMax = std::max(rightMax, height[r]);
            water += rightMax - height[r];
            --r;
        }
    }
    return water;
}

// Alternative 1: prefix/suffix max arrays. Same formula, O(n) extra space,
// the most obvious version – say it first, then improve to two pointers.
int trapPrefixMax(std::vector<int>& height) {
    int n = static_cast<int>(height.size());
    if (n == 0) return 0;
    std::vector<int> leftMax(n), rightMax(n);
    leftMax[0] = height[0];
    for (int i = 1; i < n; ++i) leftMax[i] = std::max(leftMax[i - 1], height[i]);
    rightMax[n - 1] = height[n - 1];
    for (int i = n - 2; i >= 0; --i) rightMax[i] = std::max(rightMax[i + 1], height[i]);
    int water = 0;
    for (int i = 0; i < n; ++i) water += std::min(leftMax[i], rightMax[i]) - height[i];
    return water;
}

// Alternative 2: monotonic (decreasing) stack of indices. When a taller bar
// arrives, each popped bar is the floor of a horizontal strip bounded by the
```

```
// new bar and the next bar left on the stack. Fills water "layer by layer".
int trapMonotonicStack(std::vector<int>& height) {
    std::vector<int> stack; // indices, heights decreasing bottom->top
    int water = 0;
    for (int i = 0; i < static_cast<int>(height.size()); ++i) {
        while (!stack.empty() && height[stack.back()] < height[i]) {
            int floor = stack.back();
            stack.pop_back();
            if (stack.empty()) break; // no left wall
            int left = stack.back();
            int width = i - left - 1;
            int depth = std::min(height[left], height[i]) - height[floor];
            water += width * depth;
        }
        stack.push_back(i);
    }
    return water;
}

int main() {
    auto check = [](std::vector<int> h, int expected) {
        assert(trap(h) == expected);
        assert(trapPrefixMax(h) == expected);
        assert(trapMonotonicStack(h) == expected);
    };
    check({0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1}, 6);
    check({4, 2, 0, 3, 2, 5}, 9);
    check({}, 0); // empty
    check({5}, 0); // single bar
    check({1, 2, 3, 4}, 0); // monotonic: nothing trapped
    check({4, 3, 2, 1}, 0);
    check({3, 0, 0, 3}, 6); // flat basin
    std::cout << "OK 08_trapping_rain_water.cpp\n";
    return 0;
}
```

What is new:

- `int l = 0, r = static_cast<int>(height.size()) - 1;` declares two ints on one line. For an empty vector `r` becomes -1, `l < r` is false at once, and the answer is 0 with no special case.
- `leftMax = std::max(leftMax, height[l]);` is `left_max = max(left_max, height[l])`. Needs `#include <algorithm>`.
- `water += leftMax - height[l];` is never negative because `leftMax` was just updated to include `height[l]`.
- In `trapPrefixMax`, `std::vector<int> leftMax(n), rightMax(n);` makes two vectors of `n` zeros, and the loop `for (int i = n - 2; i >= 0; --i)` walks right to left.
- In `trapMonotonicStack`, `if (stack.empty()) break;` after a pop guards against there being no left wall; without it `stack.back()` would read an empty vector, which is undefined behaviour in C++.
- `check` in `main` is a lambda that runs all three versions on the same input and asserts they agree.

Walk through the example. `height = [4, 2, 0, 3, 2, 5]`, two pointers.

<code>l=0 r=5</code>	<code>height[0]=4 < height[5]=5</code>	<code>leftMax=4</code>	<code>water += 4-4 = 0</code>	<code>-> 0</code>	<code>l=1</code>
<code>l=1 r=5</code>	<code>height[1]=2 < 5</code>	<code>leftMax=4</code>	<code>water += 4-2 = 2</code>	<code>-> 2</code>	<code>l=2</code>
<code>l=2 r=5</code>	<code>height[2]=0 < 5</code>	<code>leftMax=4</code>	<code>water += 4-0 = 4</code>	<code>-> 6</code>	<code>l=3</code>
<code>l=3 r=5</code>	<code>height[3]=3 < 5</code>	<code>leftMax=4</code>	<code>water += 4-3 = 1</code>	<code>-> 7</code>	<code>l=4</code>
<code>l=4 r=5</code>	<code>height[4]=2 < 5</code>	<code>leftMax=4</code>	<code>water += 4-2 = 2</code>	<code>-> 9</code>	<code>l=5</code>
<code>l=5 r=5</code>	<code>l < r</code> is false: stop. <code>water = 9</code>				

Every step moved the left pointer because the bar under the right pointer (5) was taller than every left bar. That is the whole justification: with a 5 waiting on the right, each left bar's water is capped by the left side only. Compare with the drawing: column 1 has 2 squares of water, column 2 has 4, column 3 has 1, column 4 has 2.

Complexity. Two pointers: one pass, $O(n)$ time, $O(1)$ extra memory. Two arrays: $O(n)$ time and $O(n)$ memory. Monotonic stack: $O(n)$ time (each index pushed and popped once), $O(n)$ memory in the worst case.

Robotics. The leftMax/rightMax formulation is a one-dimensional "flood fill to the rim": the fill depth of a terrain profile for traversability, or the puddle volume in a height map from a stereo camera. The 2D version (LeetCode 407) is a priority-queue BFS from the border cells, which is a wavefront over a cost map.

Follow-ups you may get. - *Why is moving the pointer on the shorter side safe?* If `height[l] < height[r]`, then `rightMax >= height[r] > height[l]`, so the level above bar `l` is limited by the left side only, and `leftMax` already knows it. - *2D version (LeetCode 407)?* Min-heap seeded with all border cells; pop the lowest, any lower neighbour traps `level - h`, push neighbours with `max(level, h)`. - *Container With Most Water (LeetCode 11)?* Also two pointers moving inward, but there you move the shorter side to look for a bigger area; no water is "trapped".

Part C — Check yourself

Answer each in plain words, from memory. If you cannot, reread the section named.

1. In a heap stored in an array, where are the children and the parent of index `i`? Check your formula on the 7-element example in A1. (A1)
2. Push 45 into `[50, 30, 40, 10, 20, 35, 15]` and write the array after each swap. (A1)
3. Why does a heap pop cost about $\log n$ steps and not n ? (A1)
4. Write the C++ line that declares a priority queue of ints with the smallest on top. What does the same `std::greater` do when you give it to `std::sort` instead? (A1)
5. You want the 3 largest of a million numbers arriving one at a time. Min-heap or max-heap? Why that one? (A2)
6. What does `std::nth_element` promise about the elements before and after the chosen position, and what does it not promise? (A2)
7. In the sliding window maximum, why is it safe to throw out an older, smaller value when a bigger one arrives? Why does the deque store indices and not values? (A3)
8. Run next-greater-element on `[5, 3, 4, 6]` by hand and show the stack after each step. (A3)
9. On the 5-node tree of A4, write the pre-order, in-order and post-order lists. Which one do you use to delete a tree, and why? (A4)
10. Write the level-order loop from memory. What goes wrong if you write `for (int i = 0; i < q.size(); ++i)` instead of taking a snapshot? (A4, Problem 5)
11. In the LCA recursion, what are the three things a node can return, and what does the parent do with each combination? (A5, Problem 6)
12. Why is `root == p` (comparing pointers) right and `root->val == p->val` wrong? (Problem 6)
13. In the LRU cache, why do we need both a hash map and a linked list? What does each one do in one step that the other cannot? (A6)
14. What does `std::list::splice` do, why is it constant time, and why do the bookmarks stored in the map stay valid after it? (A6, Problem 7)
15. In Trapping Rain Water, write the one-line formula for the water above bar `i`, then explain in one sentence why the two-pointer version can settle the shorter side. (Problem 8)